Содержание

1. Микросервисы 6

1.1 Что такое микросервис и зачем вообще дробить систему на сервисы? 6

1.2 Когда монолит лучше микросервисов? 7

1.3 Какие главные минусы монолита в долгой перспективе? 7

1.4 Какие ключевые плюсы микросервисов для бизнеса и команды? 8

1.5 Что такое «границы сервиса» и как их правильно проводить? 8

1.6 Как Conway’s Law влияет на дизайн микросервисов? 9

1.7 Что такое «владение данными» (data ownership) и зачем оно нужно? 9

1.8 Почему в микросервисах часто выбирают eventual consistency вместо сильной консистентности? 10

1.9 Какие основные риски при миграции с монолита на микросервисы? 10

1.10 Что такое контракт сервиса и как обеспечивать обратную совместимость API? 11

1.11 Как вы реализуете идемпотентность в публичных операциях сервиса? 12

1.12 Какие подходы к версионированию API считаешь практичными на проде? 12

1.13 Как проектировать пагинацию и фильтрацию, чтобы API были устойчивыми к росту данных? 13

1.14 Какие стратегии rate limiting и почему они важны в распределённой системе? 13

1.15 Чем отличаются синхронные и асинхронные взаимодействия сервисов по устойчивости? 14

1.16 Как подходить к медленным запросам на уровне архитектуры? 15

1.17 Какие типичные анти-паттерны микросервисов встречал и как их избегать? 15

1.18 Что считать «готовностью к эксплуатации» (operational readiness) нового сервиса? . . 16

2. Kafka 17

2.1 Что такое Apache Kafka и зачем её используют в микросервисной архитектуре? 17

2.2 Чем топик отличается от партиции и зачем нужны ключи сообщений? 17

2.3 Как гарантируется порядок сообщений и когда он теряется? . . . 18

2.4 Что такое consumer group и как работает ребалансировка? . . . 19

2.5 Чем отличаются стратегии назначения партиций (range, round-robin, sticky)? . . . 20

2.6 Что такое replication factor и ISR, зачем это нужно? . . . 20

2.7 Для чего параметр min.insync.replicas и как он связан с надёжностью? . . . 21

2.8 Как acks=0/1/all влияет на доставку и производительность? . . . 22

2.9 Как работают ретраи и идемпотентный продюсер? (что это разобрать по отдельности) . . . 23

2.10 Что такое транзакции в Kafka и как ими добиться exactly-once? . . . 24

2.11 acks и семантики доставки это одно и то же (acks all == exactly-once)? . . . 25

2.12 Когда выбирать compaction, а когда delete-retention, и почему? . . . 25

2.13 Что такое DLQ, для чего это использовать? . . . 26

2.14 Какие метрики Kafka важно мониторить в проде? . . . 26

2.15 Как контролировать размер сообщений, компрессию и батчирование (linger, batch.size)? . . . 27

2.16 Что такое KRaft и чем он заменил ZooKeeper? . . . 28

2.17 Как подходить к эволюции схем (Avro/Protobuf/JSON Schema) и режимам совместимости? И нужно ли? Если да, то когда? . . . 28

2.18 Как реализовать «в разумных пределах» идемпотентность потребителя? И нужно ли? . . . 29

2.19 Какие типичные проблемы при высоком трафике и как их диагностировать? . . . 30

2.20 Когда лучше вовсе не использовать Kafka и чем её заменить? . . . 31

3. Kubernetes **32**

3.1 Зачем Kubernetes и какие задачи он решает для микросервисов? . . . 32

3.2 Что такое Pod и почему это минимальная единица деплоя? . . . 32

3.3 Для чего нужны Deployment и ReplicaSet, чем отличаются? . . . 33

3.4 Какие типы Service (ClusterIP/NodePort/LoadBalancer) и когда их использовать? . . . 34

3.5 Что такое ConfigMap и Secret, чем они различаются по назначению? . . . 34

3.6 Что делают liveness/readiness/startup-probes и как настраивать? . . . 35

3.7 Что такое requests/limits, зачем они нужны и как влияют на HPA? . . . 36

3.8 Для чего нам вообще пробы? . . . 37

3.9 В чём разница между StatefulSet, Deployment и DaemonSet? . . . 38

3.10 Как работают PV/PVC/StorageClass и когда нужен StatefulSet? . . . 38

3.11 Что такое Ingress и чем он отличается от Gateway-подходов? . . . 39

3.12 Как работают taints/tolerations и node/pod affinity? . . . 40

3.13 Что такое NetworkPolicy и как ограничивать сетевые потоки? . . . 41

3.14 Как проводить безопасные релизы: rolling update параметры, maxSurge/maxUnavailable? . . . 42

3.15 Как организовать секреты: KMS, sealed-secrets, внешние хранилища? . . . 42

3.16 Какие ключевые метрики и эвенты смотреть при инциденте? . . . 43

3.17 Как дебажить «под висит/крашится» (kubectl describe/logs, events)? . . . 44

3.18 Что учитывать при автоскейлинге (HPA/VPA, метрики, холодный старт)? . . . 44

3.19 Что такое Ingress, istio и че это вообще значит все? . . . 45

3.20 В чем разница между k8s и Kubernetes (минимум 10 отличий)? . . . 46

4. REST, gRPC и Kafka **46**

4.1 Почему не стоит всё делать только REST, и когда он уместен? Нормально ли базировать всю систему на REST? . . . 46

4.2 В чём преимущества gRPC по сравнению с REST для внутрянки? . . . 47

4.3 Когда выбирать асинхронность через Kafka вместо синхронного RPC? . . . 47

4.4 Как проектировать контракты: JSON vs Protobuf, бинарные протоколы? (API First, CodeFirst подходы) . . . 48

4.5 Какие плюсы/минусы стриминга в gRPC (server/client/bidi-streaming)? . . . 49

4.6 Как обеспечивать обратную совместимость при изменении контрактов? . . . 49

4.7 Как выбирать паттерн коммуникации: запрос-ответ, fire-and-forget, pub/sub? . . . 50

4.8 Как решать проблему каскадных таймаутов в синхронных цепочках? . . . 51

4.9 Что важно учесть для идемпотентности при повторной доставке? 52

4.10 Как организовать трассировку сквозь REST/gRPC/Kafka (trace-идентификаторы)? (OpenTelemetry + Jaeger) 52

4.11 Как бороться с «шумом» и потерями при массовых ретраях? 53

4.12 Когда уместен API Gateway, а когда BFF, и как это влияет на протоколы? 54

5. Паттерны микросервисов **55**

5.1 Что такое API Gateway и зачем он нужен? (и остальные паттерны микросервисов. ВСЕ!) 55

5.2 Чем BFF отличается от «просто» API Gateway и когда его вводить? 56

5.3 Что такое Circuit Breaker и когда он срабатывает? 57

5.4 Для чего нужны Bulkhead, RateLimiter и Timeouts вместе? 58

5.5 Что такое Retry-стратегии и как избежать thundering herd? 58

5.6 Как работает паттерн Outbox и чем он помогает против «двух записей»? 59

5.7 Что такое Transactional Inbox/Outbox и в чём их различие? 60

5.8 Когда уместен Event Sourcing и какие у него подводные камни? 61

5.9 Что такое CQRS и зачем разделять чтение и запись? 62

5.10 Двухфазная фиксация (2PC): как работает и почему редко подходит в микросервисах? 62

5.11 Saga: оркестрация vs хореография — плюсы/минусы, где какую взять? 63

5.12 TCC (Try-Confirm/Cancel): чем отличается от Saga и когда проще внедрить? 64

5.13 Как проектировать компенсирующие операции и хранить их контекст? 65

5.14 Как выбирать ключи идемпотентности и TTL для хранения факта выполнения? 65

5.15 Как решать дедупликацию событий при «ровно-один-раз» бизнес-уровнем? 66

5.16 Когда применять «strangler-fig» для постепенного распила монолита? 67

6. Наблюдаемость **68**

6.1 Для чего нам это все дело? 68

6.2 Какие поля логировать обязательно (traceId, spanId, user, важные ключи домена)? . . . 68

6.3 Как устроена цепочка ELK: кто и за что отвечает (сбор, парсинг, хранение, поиск)? . . 69

6.4 Как проектировать индексы и ILM-политику, чтобы не «сжечь» кластер? 70

6.5 Что такое Spring Actuator и какие эндпоинты включать в проде? 71

6.6 Как Micrometer/Prometheus собирают метрики из Spring Boot? 72

6.7 Какие типы метрик Prometheus (counter/gauge/histogram/summary) и для чего каждый? 72

6.8 Для чего нужна графана, метрики в ней, алерты? 74

7. Spring Cloud **74**

7.1 Для чего Spring Cloud Config и как безопасно хранить секреты/ключи? 74

7.2 Для чего нам вообще Config как отдельный сервис? 75

7.3 Как настроить Spring Cloud Gateway: маршрутизация, фильтры, троттлинг? 76

7.4 Как использовать OpenFeign: таймауты, ретраи, декодеры ошибок, fallback? 77

7.5 Как включать CircuitBreaker (Resilience4j) и какие параметры критичны? 78

7.6 Как под капотом работает Spring Cloud Api Gateway? Что принимает, что возвращает . 79

1. Микросервисы

1.1 Что такое микросервис и зачем вообще дробить систему на сервисы?

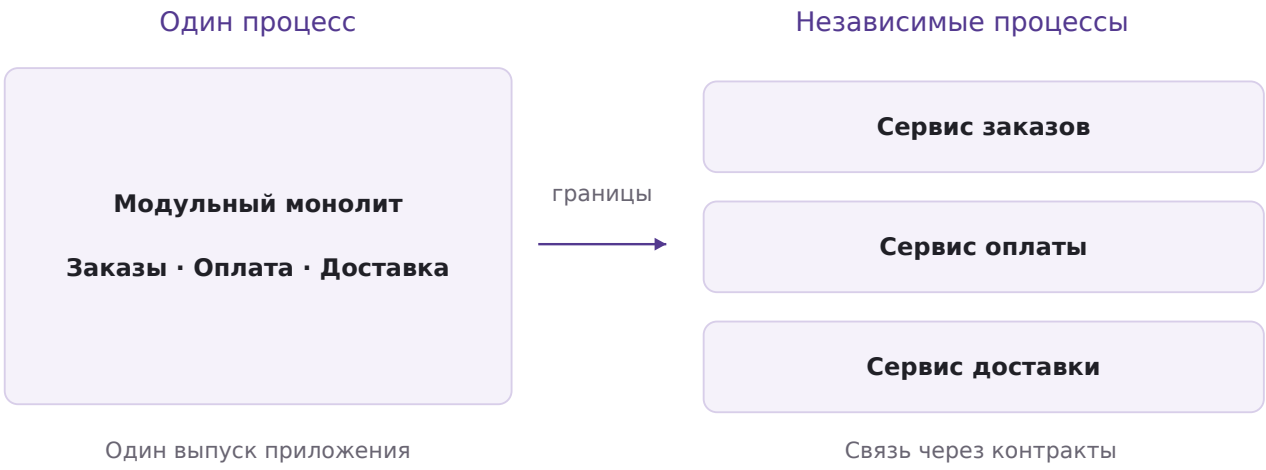
Представим приложение магазина. В нем есть заказы, оплата, склад и доставка. Пока все это находится в одном приложении и выпускается одной сборкой, перед нами монолит. Внутри могут быть хорошие модули и четкие интерфейсы. Монолит не означает беспорядок.

Микросервис отвечает за отдельную бизнес-возможность и может развертываться, то есть запускаться или обновляться в рабочем окружении, независимо от остальных сервисов. Например, сервис доставки сам хранит заявки на доставку, меняет их состояние и предоставляет другим системам API. Это набор операций, которые другие программы могут вызвать, например создание заявки или проверку ее статуса.

Он живет в отдельном процессе. Поэтому вызов другого сервиса проходит по сети. В отличие от обычного вызова Java-метода, такой вызов может задержаться, оборваться или завершиться на сервере уже после того, как клиент перестал ждать.

Дробление полезно, когда частям системы нужны разные темпы изменений, разная нагрузка или самостоятельные команды. Каталог можно масштабировать под чтение, обработку платежей развивать по своим правилам, а релиз доставки выпускать без пересборки каталога.

Размер кода здесь вторичен. Сервис из трех классов, который не может работать и выпускаться без пяти соседей, еще не дает независимости. Начинать стоит с вопроса о том, какая ответственность должна изменяться самостоятельно, а не с желания получить больше приложений.



1.2 Когда монолит лучше микросервисов?

Небольшая команда делает новый продукт. Требования меняются, границы предметной области еще неясны, нагрузка умеренная. В такой ситуации **модульный монолит часто дает более быстрый и управляемый старт**. У него один процесс развертывания, простые локальные вызовы и обычные транзакции в базе.

Модульность означает, что код заказов не лезет напрямую во внутренние классы оплаты. У модулей есть публичные интерфейсы, а правила зависимостей проверяются. Это позволяет поддерживать порядок без сетевого разделения.

Монолит удобен и тогда, когда операция действительно требует одновременного изменения тесно связанных данных. Списать остаток и записать проводку в одной локальной транзакции проще, чем согласовывать несколько сервисов.

Число пользователей само по себе ничего не решает. Монолит можно запускать в нескольких экземплярах за балансировщиком, а тяжелые задачи выполнять отдельными фоновыми обработчиками. Если узкое место находится в запросе к базе, нарезка приложения на сервисы его не исправит.

Переход к микросервисам имеет смысл, когда ограничения общего релиза, общего масштабирования и общей ответственности уже измеримы. Цена нового подхода включает мониторинг, сеть, доставку событий, совместимость контрактов и дежурство по нескольким приложениям.

1.3 Какие главные минусы монолита в долгой перспективе?

У большого монолита часто появляется **связанность изменений**. Команда меняет расчет доставки, но выпускает весь продукт. Чем больше общий набор зависимостей и регрессионных проверок, тем сложнее понять, затронет ли релиз чужую функциональность.

Второй риск связан с границами кода. Общая база и прямой доступ ко всем классам упрощают быстрые правки. Со временем платежный модуль может начать зависеть от внутренней структуры каталога, а изменение таблицы потребует согласования нескольких команд.

Третий риск касается ресурсов и отказов. Утечка памяти в одной части процесса может остановить все приложение. Если нагружен только поиск, горизонтальное масштабирование обычно добавляет экземпляры всего монолита. Это бывает дороже отдельного масштабирования поиска.

Но эти проблемы не возникают автоматически. Четкие модули, ограничение зависимостей, оптимизация сборки, отдельные пулы ресурсов и хорошее тестирование заметно отодвигают их. И наоборот, если сохранить связанность после разделения, получится та же проблема с дополнительными сетевыми отказами.

Поэтому полезно назвать конкретное ограничение. «Релиз согласуют четыре команды» уже описывает проблему. «Монолит старый» еще ничего не объясняет.

1.4 Какие ключевые плюсы микросервисов для бизнеса и команды?

Для команды основной выигрыш состоит в возможности самостоятельно менять и выпускать свою область. Если контракт оплаты остается совместимым, команда оплаты может развернуть новую версию, не дожидаясь релиза доставки.

Для бизнеса это может сократить время между решением и работающей функцией. Отдельный сервис также позволяет увеличивать ресурсы именно там, где вырос спрос. Массовое чтение каталога не обязано требовать столько же дополнительных экземпляров платежной логики.

Изоляция помогает ограничивать последствия отказа. Магазин может продолжить показывать каталог, когда отправка уведомлений временно не работает. Но для этого уведомления должны быть действительно отделены, а приложение должно уметь жить без немедленного ответа от них.

У каждой команды появляется понятная ответственность за данные, контракт, эксплуатацию и стоимость своей части продукта. Разные технологические решения допустимы там, где оправданы задачей, хотя слишком большое разнообразие увеличивает расходы на поддержку.

Все эти плюсы являются результатом правильных границ и организации работы. Сам факт запуска нескольких Spring Boot приложений их не обеспечивает. Если любой релиз требует одновременно обновить всю систему, независимость пока не достигнута.

1.5 Что такое «границы сервиса» и как их правильно проводить?

Граница сервиса определяет, какие бизнес-правила и данные находятся под одной ответственностью. Внутри границы сервис может принимать решения самостоятельно. За ее пределами он работает через контракт.

В магазине наличие товара и его описание связаны, но решают разные задачи. Каталог отвечает за то, что мы продаем, склад за то, сколько можно отгрузить. Если же разнести каждую таблицу по отдельному сервису, оформление заказа превратится в десятки сетевых обращений.

Начинают с бизнес-процессов, терминов и **инвариантов**. Инвариант означает правило, которое нельзя нарушить. Например, зарезервированное количество не должно превышать доступный остаток. Данные и операции, необходимые для такого правила, полезно держать в одной транзакционной границе.

При проектировании по предметной области, Domain-Driven Design, используют понятие bounded context. По-русски его называют ограниченным контекстом. Это область, внутри которой термины и модель имеют согласованный смысл. В оплате «клиент» может означать плательщика, а в доставке получателя. Одна универсальная модель для всех областей часто мешает.

После выделения границ проверяют реальные взаимодействия. Если два сервиса постоянно читают внутренние таблицы друг друга, требуют общего релиза и делают много мелких синхронных вызовов, границу стоит пересмотреть. Ограниченный контекст помогает определить смысл модели, но не требует механически создавать отдельный сервис на каждый термин.

1.6 Как Conway’s Law влияет на дизайн микросервисов?

Conway’s Law описывает наблюдение о том, что устройство системы отражает коммуникацию людей, которые ее создают. Если за отдельные подсистемы отвечают изолированные группы, в архитектуре часто появляются похожие разделения.

Представим три команды. Одна делает все базы, вторая всю бизнес-логику, третья все API. Даже простая функция проходит через три очереди согласований. Получившиеся сервисы легко начинают повторять технические слои вместо самостоятельных бизнес-областей.

Если команда отвечает за оформление заказа целиком, ей проще держать вместе API, правила заказа, его данные и эксплуатацию. Тогда большая часть изменений решается внутри команды, а внешние зависимости проходят через устойчивые контракты.

Это не физический закон и не требование подгонять архитектуру под любое текущее штатное расписание. Структуру команд можно менять вместе с границами системы. Например, сначала выделяют область заказов и формируют команду, способную самостоятельно ее развивать. Такой подход называют обратным маневром Конвея.

Практическая проверка простая. Если обычная задача требует согласования пяти команд, независимость сервисов пока ограничена организацией работы. Добавление брокера сообщений само по себе это не исправит.

1.7 Что такое «владение данными» (data ownership) и зачем оно нужно?

Владелец данных определяет правила их изменения и отвечает за их смысл. Остальные сервисы не обновляют его таблицы напрямую. Если статус платежа принадлежит сервису оплаты, доставка узнает его через API или событие.

Такой порядок защищает бизнес-правила. Прямая запись в таблицу оплаты обходит проверку переходов состояний, аудит и публикацию событий. Кроме того, структура таблиц становится неявным общим контрактом, который трудно менять.

Отдельный физический сервер базы для каждого сервиса не обязателен. Несколько сервисов могут использовать один кластер PostgreSQL с отдельными схемами, учетными записями и правами. Важна логическая изоляция, хотя общий кластер сохраняет общий предел ресурсов и общую область отказа.

Чужие данные разрешается копировать в локальную модель чтения. Например, доставка хранит адрес получателя, полученный при оформлении заказа. Нужно

понимать, является ли это снимком на момент покупки или актуальной копией, которую требуется обновлять событиями.

Цена владения состоит в том, что обычный SQL JOIN, соединение строк из разных таблиц, между таблицами разных владельцев уже не является безопасной основой интеграции. Для экрана используют сборку ответа из нескольких API, отдельную таблицу для нужного экрана или аналитическое хранилище. Выбор зависит от свежести данных и допустимой задержки.

1.8 Почему в микросервисах часто выбирают eventual consistency вместо сильной консистентности?

Платеж прошел, а история заказов еще несколько секунд показывает прежний статус. Это пример eventual consistency. Изменение уже принято владельцем данных, но другие представления догоняют его позже.

Согласованность с задержкой, eventual consistency, означает сходимость копий при отсутствии новых изменений и при работающем механизме доставки и разрешения конфликтов. Она не обещает фиксированную задержку и не возникает автоматически после добавления Kafka.

В распределенной системе немедленное согласование нескольких участников требует координации. Если один участник недоступен, система должна либо ждать и отказывать в части операций, либо разрешать временные различия. Поэтому локальную операцию нередко завершают независимо, а изменения распространяют событиями.

При этом внутри сервиса может оставаться строгая транзакция. Оплата не должна дважды списывать деньги только потому, что внешний отчет обновляется с задержкой. Нужную гарантию выбирают для конкретного правила, а не для всего приложения одним словом.

Чтобы копии сошлись, нужны повторная доставка, идемпотентная обработка, контроль порядка или версии события, а также сверка пропусков. Пользователю показывают состояние обработки, а не выдают незавершенную операцию за готовую.

CAP здесь не означает «можно выбрать любые два свойства навсегда». Ограничение проявляется при сетевом разделении. Более того, сильная согласованность имеет разные формальные модели. Линеаризуемое чтение гарантирует, что после завершенной записи последующее чтение увидит ее результат или более новое значение. Сериализуемые транзакции дают результат, который можно объяснить некоторым последовательным порядком транзакций. Это разные гарантии, поэтому одного слова «сильная» для точного описания недостаточно.

1.9 Какие основные риски при миграции с монолита на микросервисы?

Главный риск миграции состоит в том, что код уже разделили, а ответственность и данные остались общими. Новый сервис читает старые таблицы, старое приложение пишет в новые, и любое изменение снова требует общего релиза.

Особенно опасна переносимая запись данных. Если временно писать и в старую базу, и в новую двумя независимыми вызовами, между ними возможен сбой. Нужны один источник истины, надежное распространение изменений и сверка результатов.

Перенос начинают с ограниченной области. Сначала выясняют ее зависимости, добавляют наблюдаемость, проверяют контракт, затем постепенно направляют на новую реализацию реальные операции. Старые и новые версии некоторое время должны сосуществовать.

Риски включают изменение задержек, потерю локальной транзакции, неожиданные ограничения сети, ошибки переноса исторических данных и недостаток эксплуатационного опыта. Откат кода не всегда откатывает уже измененные данные, поэтому план возврата проверяют отдельно.

Для безопасного перехода заранее определяют признаки успеха. Это могут быть доля ошибок, время ответа, расхождения при сверке и возможность независимо выпустить новую область. Миграция ради числа сервисов не дает полезного критерия завершения.

1.10 Что такое контракт сервиса и как обеспечивать обратную совместимость API?

Контракт описывает то, на что вправе рассчитывать потребитель. В HTTP это не только URL и JSON. Сюда входят метод, коды ответа, обязательность полей, ограничения, авторизация, смысл ошибок и поведение повторных запросов.

Если раньше сумма означала рубли, а теперь копейки, тип поля может остаться числом, но контракт уже сломан. Совместимость проверяют и по форме сообщения, и по его смыслу.

Безопасное изменение обычно расширяет возможности. Сервер начинает принимать новое необязательное поле, сохраняя прежнее поведение при его отсутствии. Но даже добавление поля в ответ способно сломать строгий клиент, поэтому допуск неизвестных полей должен быть частью договоренности.

Для удаления используют постепенный переход, который называют **expand-contract**. Сначала контракт расширяют, а после перехода клиентов убирают старую часть. Сначала добавляют новую форму, затем переводят потребителей, проверяют использование старой и только после этого убирают ее. Для данных это может включать временную поддержку двух колонок и заполнение исторических значений.

OpenAPI фиксирует структуру HTTP API, контрактные тесты проверяют ожидания конкретных клиентов. В таких тестах клиент описывает, какой запрос он отправит и какой ответ должен суметь обработать. Ни то ни другое не заменяет проверку бизнес-смысла. Совместимость означает, что старый клиент продолжает правильно выполнять свою задачу.

1.11 Как вы реализуете идемпотентность в публичных операциях сервиса?

Клиент отправил платеж, сервер его выполнил, но ответ потерялся. Клиент повторяет запрос. **Идемпотентная операция сохраняет один бизнес-эффект при повторении той же команды.** Одинаковый HTTP-ответ для всех попыток возможен как часть API, но не является единственным определением идемпотентности.

Для создания платежа клиент генерирует Idempotency-Key один раз и повторяет его вместе с тем же содержимым. На сервере ключ связывают с пользователем или организацией и видом операции. Иначе чужой ключ может пересечься с нашим.

В базе нужна уникальность этого сочетания. В одной локальной транзакции сервер занимает ключ, выполняет изменение и сохраняет результат. Параллельный запрос с тем же ключом не должен пройти проверку отдельно и повторить списание.

Для ключа сохраняют отпечаток существенных полей запроса. Это значение, рассчитанное по сумме, получателю и другим важным данным. Если тот же ключ пришел с другой суммой, сервер возвращает конфликт, а не старый результат от другого платежа. Незавершенное состояние различают с завершенным. Дубликат может получить ссылку на операцию и предложение проверить ее статус.

Если эффект находится во внешней платежной системе, локальной транзакции недостаточно. Ключ передают внешнему провайдеру, если он его поддерживает, либо используют надежный процесс со сверкой неопределенного результата. Запись «выполнено» перед внешним вызовом способна потерять операцию, а запись после вызова оставляет окно повтора.

1.12 Какие подходы к версионированию API считаешь практичными на проде?

Версию вводят, когда изменение нельзя совместить с прежним контрактом. Для публичного HTTP API понятен путь `/v1/payments`. Его видно в логах, маршрутах и документации. Передавать версию в HTTP-заголовке или в обозначении формата ответа тоже допустимо, если инфраструктура и клиенты их одинаково понимают.

Номер версии не нужен для каждого нового необязательного поля. Иначе небольшие изменения создают множество почти одинаковых API, которые придется одновременно сопровождать.

Новая несовместимая версия означает период сосуществования. Нужны владелец миграции, описание различий, наблюдение за использованием старой версии и согласованный срок ее отключения. Простая замена `/v1` на `/v2` без этой работы не решает проблему.

Для событий часто лучше развивать схему совместимо в прежнем топике. Отдельный топик новой версии оправдан, если изменились смысл события, правила обработки или формат так, что безопасного совместного чтения уже нет. Версия схемы и версия бизнес-события не обязаны совпадать.

Практичный выбор определяется тем, кто контролирует клиентов. Внутренняя команда может согласовать миграцию быстрее. Мобильное приложение пользователя способно месяцами оставаться на старой версии, поэтому его контракт требует более длинной поддержки.

1.13 Как проектировать пагинацию и фильтрацию, чтобы API были устойчивыми к росту данных?

API не должно возвращать все заказы сразу. Даже если сейчас их сто, позже ответ может перестать помещаться в память сервера и клиента. Поэтому ограничивают размер страницы, набор разрешенных фильтров и стоимость сортировки.

Пагинация со смещением, offset-пагинация, выглядит понятно. Пропустить первые 100 записей и взять следующие 20. Но база все равно выполняет работу по поиску и пропуску строк. При больших смещениях это дорого. Вставки и удаления между запросами могут давать повторы и пропуски.

Пагинация с курсором продолжает чтение после последнего известного значения. Курсор хранит место, на котором закончилась предыдущая страница. Для устойчивого порядка одного времени недостаточно, поскольку несколько заказов могут иметь одинаковую дату. Добавим уникальный id.

POSTGRESQL

```
SELECT id, created_at, status
FROM orders
WHERE customer_id = :customerId
  AND (created_at, id) < (:lastTime, :lastId)
ORDER BY created_at DESC, id DESC
LIMIT :pageSize;
```

Это фрагмент запроса для PostgreSQL с параметрами приложения. Его поддерживает индекс по `(customer_id, created_at DESC, id DESC)`. Первая страница выполняется без условия курсора. В курсор включают обе последние величины и параметры выборки, а его содержимое проверяют или подписывают.

Сортируемые поля желательно не менять во время обхода. Иначе запись может перескочить между страницами. Для строгого экспорта нужен снимок или отдельный процесс выгрузки. Точное общее количество тоже может быть дорогим, поэтому иногда достаточно признака следующей страницы. Авторизация применяется на каждой странице независимо от курсора.

1.14 Какие стратегии rate limiting и почему они важны в распределённой системе?

Ограничение частоты запросов, rate limiting, ограничивает число операций за время. Он защищает сервис от случайного цикла клиента, всплеска нагрузки и несправедливого расхода общего ресурса одним потребителем.

Подход	Как считает	Что учитывать
Fixed window	Счетчик в пределах интервала	На границе интервалов возможен двойной всплеск
Sliding window	Учитывает последнее скользящее окно	Точность требует памяти или приближений
Token bucket	Токены пополняются с заданной скоростью	Разрешает ограниченный короткий всплеск
Leaky bucket	Выпускает поток с ограниченной скоростью	Очередь и время ожидания тоже нужно ограничить

Для публичного API лимит часто задают на клиента, организацию или ключ доступа. Организацию, данные и настройки которой нужно отделить от остальных клиентов, часто называют tenant. Ограничение по IP подходит не всегда, потому что за одним адресом могут находиться тысячи людей.

Локальный счетчик прост и доступен при отказе Redis, но у пяти экземпляров сервиса получится пять отдельных лимитов. Общий счетчик точнее соблюдает глобальную квоту, зато добавляет зависимость. При его отказе заранее выбирают, отклонять запросы или временно разрешать их с локальной защитой.

Для HTTP превышение квоты обычно выражают ответом 429 и при возможности Retry-After. Ограничение частоты не заменяет ограничение параллельности. Десять очень долгих запросов способны занять больше ресурсов, чем тысяча коротких.

1.15 Чем отличаются синхронные и асинхронные взаимодействия сервисов по устойчивости?

При синхронном взаимодействии вызывающий ждет ответ в рамках своей операции. Если оформление заказа обращается к оплате, задержка оплаты непосредственно входит во время ответа клиенту. Отказ зависимости может остановить весь путь.

При асинхронном взаимодействии отправитель передает работу в надежное промежуточное хранилище, а получатель выполняет ее позже. Краткий простой получателя не обязан мешать принять сообщение, пока у брокера есть доступность и место.

Но очередь не устраняет нагрузку. Она превращает ее в отставание. Если обработчик постоянно медленнее входящего потока, очередь будет расти до ограничений хранения. Нужны наблюдение за возрастом сообщений, обратное давление и политика отказа при переполнении.

Синхронный вызов удобен, когда результат нужен сейчас. Например, нельзя подтвердить резерв товара, не узнав, был ли резерв создан. Асинхронная модель подходит, когда API может вернуть принятое к обработке задание и дать проверить состояние позже.

Надежность выбирается для всей цепочки. Ответ «сообщение принято в память процесса» не равен надежному приему. Асинхронность требует сохранения команды, правил повторной обработки и понятного пользователю конечного статуса.

1.16 Как подходить к медленным запросам на уровне архитектуры?

Сначала выясняют, где тратится время. Медленный SQL, очередь за соединением, удаленный сервис, сериализация большого ответа и занятый CPU требуют разных решений. Трассировка показывает путь запроса, а метрики помогают понять, единичная это задержка или системная.

Если время уходит в базу, проверяют план запроса, индексы, объем выборки и количество обращений. Если в удаленные вызовы, убирают лишние переходы и проверяют, какие запросы действительно независимы и могут выполняться параллельно.

Кэш полезен, если допустимы устаревшие данные и понятны правила обновления. Он не должен скрывать проблему неопределенным обещанием «будет быстрее». Для очень часто запрашиваемого ключа и массового истечения времени хранения кэша нужен контроль одновременного заполнения.

Долгую задачу можно оформить как ресурс операции. API сохраняет задание, возвращает 202 и его идентификатор, затем фоновый обработчик выполняет работу, а клиент читает статус. Простое продолжение задачи в памяти после ответа ненадежно при перезапуске.

Еще до оптимизации задают предел ожидания и ограничивают число одновременно выполняемых работ. Увеличенный таймаут иногда только увеличивает очередь и расход памяти. Результат проверяют по задержкам и нагрузке на зависимые системы. Например, p95 показывает время, в которое укладываются 95% запросов, а p99 показывает границу для 99%. Это помогает заметить долгие запросы, которые скрывает среднее значение.

1.17 Какие типичные анти-паттерны микросервисов встречал и как их избегать?

Частый антипаттерн называют распределенным монолитом. Приложений много, но они выпускаются вместе, используют общие таблицы и требуют непрерывной доступности друг друга. Исправляют это пересмотром границ и контрактов, а не добавлением еще одного промежуточного слоя.

Другой пример состоит в слишком мелких сервисах. Один получает пользователя, второй его имя, третий адрес. Чтобы показать экран, приходится делать множество вызовов. Полезнее дать сервису законченную бизнес-ответственность или собрать модель чтения для экрана.

Общая библиотека со всеми DTO и сущностями тоже связывает команды. DTO являются объектами для передачи данных между частями приложения. Если все сервисы используют один общий набор таких объектов, изменение одного поля

может потребовать обновить несколько приложений. Общие технические библиотеки допустимы, но бизнес-модели разных областей не стоит автоматически объединять.

Опасны бесконечные ретраи, отсутствие таймаутов и синхронные циклы вызовов. Они превращают локальный сбой в общий. В асинхронной системе похожую проблему создает событие, которое фактически заставляет все сервисы синхронно согласовывать один процесс.

Еще один признак проблемы появляется, когда сервис создан, но никто не отвечает за его алерты, стоимость и восстановление. Независимое развертывание требует самостоятельной эксплуатационной ответственности.

1.18 Что считать «готовностью к эксплуатации» (operational readiness) нового сервиса?

Сервис готов к эксплуатации, когда команда может безопасно его выпустить, заметить сбой и восстановить работу. Одного успешного ответа эндпоинта недостаточно. Эндпоинт является операцией API по определенному адресу. Он может отвечать на проверку состояния, даже если важная для пользователя операция не работает.

У сервиса должны быть понятный владелец и рабочий путь обращения при инциденте. Для важных операций определяют SLI и SLO. SLI является измеряемым показателем, например долей успешных платежей. SLO задает его целевое значение за выбранный период.

Необходимы структурированные логи, метрики, трассировка и алерты по пользовательским проблемам. Проверяют поведение при недоступности базы, брокера и внешнего API, а также лимиты ресурсов и корректное завершение запросов при остановке.

Релиз должен учитывать совместимость старой и новой версии, миграции базы и возможность возврата. Для сервиса с данными отдельно нужны резервные копии и проверенное восстановление. Наличие файла резервной копии не доказывает, что из него можно поднять систему.

Полезный практический критерий выглядит так. Другой дежурный разработчик по инструкции способен понять, что сломалось, ограничить ущерб и выполнить восстановление без автора сервиса. Для этого инструкция должна содержать реальные команды и признаки результата, а не только архитектурную схему.

2. Kafka

2.1 Что такое Apache Kafka и зачем её используют в микросервисной архитектуре?

Заказ создан. Об этом должны узнать склад, уведомления и аналитика. Если сервис заказов вызывает их всех напрямую, его работа зависит от их доступности. Kafka позволяет записать событие в общий надежный журнал, а каждому потребителю читать его в своем темпе.

Apache Kafka хранит поток событий и позволяет приложениям читать его независимо друг от друга. Сервер Kafka называют брокером. Приложение, которое отправляет сообщения, называют продюсером, producer. Приложение, которое читает их, называют консьюмером, consumer. Консьюмер сам запрашивает новые записи у брокера. Kafka не вызывает метод или HTTP-эндпоинт консьюмера.

Записи группируются в топики. Топик делится на партиции, каждая из которых представляет последовательный журнал. Внутри партиции у записи есть офсет, offset. Это числовая позиция записи в журнале. Например, запись 15 идет после записи 14. Одинаковые офсеты в разных партициях относятся к разным записям.

Консьюмер хранит позицию чтения. Подтверждение позиции не удаляет запись из топика. Поэтому аналитику можно запустить позже или перечитать историю после исправления ошибки, пока данные еще сохранены политикой хранения.

Kafka полезна для независимых реакций на события, интеграции систем, сбора изменений данных и потоковых вычислений. Она дает буфер между производством и обработкой данных, но требует администрирования, контроля отставания и проектирования повторной обработки. Нужный пользователю немедленный ответ обычно продолжает возвращаться через HTTP или удаленный вызов метода, RPC.

2.2 Чем топик отличается от партиции и зачем нужны ключи сообщений?

Топик можно представить как название потока, например `payments`. Партиции являются его отдельными последовательными журналами. Запись попадает в одну партицию, а не во все сразу. Копии этой партиции на других брокерах нужны для отказоустойчивости и называются репликами.

Партиции позволяют делить работу между несколькими консьюмерами. Четыре партиции можно назначить четырем консьюмерам одной группы. Внутри группы каждую партицию в каждый момент обрабатывает один назначенный участник. Другая группа может независимо читать те же партиции.

Ключ сообщения помогает выбрать партицию. При стандартном распределении ключ хешируется. Одинаковый `orderId` при неизменном числе партиций и одинаковой

сериализации ключа попадает в одну партицию. Это позволяет сохранить последовательность событий одного заказа.

```java
new ProducerRecord<String, String>(
    "payments", "order-42", "PAYMENT_ACCEPTED"
);
```

Это фрагмент Java-кода с классом `org.apache.kafka.clients.producer.ProducerRecord` из kafka-clients 4.1. Он создает запись, а отправку выполняет KafkaProducer. Ключ здесь равен `order-42`, значение содержит учебное текстовое событие.

Если ключ отсутствует, продюсер может выбирать партицию так, чтобы собрать больше записей в один пакет и отправить их вместе. Такой пакет называют batch. Это экономит сетевые обращения, но не обеспечивает порядок событий конкретного заказа. При изменении числа партиций даже одинаковый ключ может начать попадать в другую партицию.



В каждой группе свои назначения и подтвержденные позиции

2.3 Как гарантируется порядок сообщений и когда он теряется?

Kafka хранит порядок записей **внутри одной партиции по офсету**. Между партициями общего порядка нет. Время внутри события тоже не определяет очередность автоматически. Два продюсера могут иметь разные настройки часов и в разное время доставить записи брокеру.

Чтобы события одного заказа шли последовательно, используют стабильный ключ и одинаковое правило выбора партиции. Идемпотентный продюсер умеет безопасно повторять отправку без второй копии той же записи и сохраняет порядок своих пакетов при поддерживаемых настройках. Этот механизм подробно разобран в

вопросе о повторах. Без него первый пакет может записаться позже второго, если его пришлось отправлять еще раз.

Порядок чтения и порядок завершения обработки различаются. Консьюмер получил записи с офсетами 10 и 11 по очереди, но передал их в разные потоки Java. Если обработка 11 закончилась раньше, изменения в базе тоже могут произойти не по порядку. Поэтому последовательность нужно сохранять и внутри приложения, а подтверждать можно только непрерывную уже обработанную часть партиции.

Отдельный топик для повторных попыток и DLQ, топик для сообщений с неразрешенными ошибками, способны нарушить порядок бизнес-операций. Событие об оплате ушло на повтор, а последующее событие об отмене уже обработалось. Если такой сценарий недопустим, обработку ключа блокируют до разрешения ошибки или проверяют версию и допустимость перехода состояния.

При увеличении числа партиций планируют переход отдельно. Один и тот же хеш ключа может выбрать другую партицию. Нельзя просто считать, что одинаковый ключ навсегда означает один и тот же журнал.

2.4 Что такое consumer group и как работает ребалансировка?

Группа консьюмеров, consumer group, объединяет приложения, которые вместе выполняют одну задачу. Два экземпляра обработчика доставки с одним `group.id` делят работу. Аналитика использует другой `group.id` и читает те же сообщения независимо от доставки.

Для каждой пары «группа и партиция» сохраняется подтвержденная позиция чтения, committed offset. После сбоя группа продолжает с этой позиции. Если запись 15 обработана успешно, обычно сохраняют 16, номер следующей записи. При этом сами сообщения из топика не удаляются.

Ребалансировка перераспределяет партиции между участниками группы. Она нужна, когда консьюмер подключился, отключился, изменил подписку или когда изменилось число партиций. Координатор на брокере следит за группой. Участники регулярно отправляют ему сигналы о своей доступности, heartbeat. Метод `poll()` получает следующую порцию записей. Если Java-приложение слишком долго не вызывает его, срабатывает ограничение `max.poll.interval.ms`, и владение партициями может быть передано другому участнику.

Во время передачи партиции прежний консьюмер должен перестать выполнять работу за ее пределами владения. Результат незавершенной обработки и неподтвержденные позиции чтения требуют внимания. После сбоя записи могут прийти снова, поэтому бизнес-эффект защищают идемпотентностью.

В прежнем протоколе группы, classic, перераспределение могло приостанавливать обработку всей группы. Постепенная передача партиций позволяет сократить такие паузы. Начиная с Kafka 4.0 новый протокол групп доступен как стабильная возможность. В клиентах 4.1 его включают через `group.protocol=consumer`. Распределение партиций и часть настроек сигналов доступности переходят на брокер. Поэтому прежнюю конфигурацию клиента нельзя переносить без проверки.

2.5 Чем отличаются стратегии назначения партиций (range, round-robin, sticky)?

Стратегия назначения, assignor, решает, какой консьюмер получит какие партиции. В прежнем протоколе Java-клиента ее задают через `partition.assignment.strategy`. Варианты ниже отличаются равномерностью распределения и количеством партиций, которые придется передать при изменении группы.

Стратегия	Принцип	Особенность
Range	Делит партиции каждого топика на последовательные диапазоны	На нескольких топиках остатки могут доставаться одним участникам
Round-robin	Обходит партиции и участников по кругу	Обычно ровнее при одинаковых подписках, но перемещает много назначений
Sticky	Старается сохранить прежние назначения и баланс	Уменьшает лишние перемещения
Cooperative sticky	Совмещает сохранение назначений с постепенной передачей	Не требует отзывать все партиции одновременно

Например, два топика имеют по три партиции, а консьюмеров два. При стратегии range первый участник может получить по две партиции каждого топика, всего четыре, а второй две. Round-robin при одинаковой подписке может распределить их как три и три.

Равное число партиций не означает равную нагрузку. Одна партиция с очень горячим ключом способна обрабатывать больше данных, чем остальные вместе. Стратегия не обязана учитывать бизнес-стоимость каждой записи.

В новом `group.protocol=consumer`, доступном начиная с Kafka 4.0, партиции распределяет сервер. При необходимости клиент выбирает одну из доступных стратегий через `group.remote.assignor`. В документации 4.1 описаны uniform, равномерное распределение, и range, распределение диапазонами. Настройка `partition.assignment.strategy` относится к прежнему протоколу и здесь не определяет результат.

2.6 Что такое replication factor и ISR, зачем это нужно?

Replication factor задает число копий партиции. При значении 3 журнал хранится на трех брокерах. Одну копию называют лидером, leader. Продюсер отправляет записи лидеру, а остальные копии, followers, догоняют его журнал. Консьюмер обычно тоже читает у лидера. Чтение у других копий возможно при специальной настройке.

ISR является набором реплик, которые достаточно синхронизированы с лидером. Сам лидер тоже входит в набор. Если другая копия слишком долго отстает

или недоступна, ее могут исключить. После восстановления и синхронизации ее возвращают. Название ISR расшифровывается как in-sync replicas.

Это не постоянный список «здоровых серверов». Реплика может кратко отставать, оставаясь в допустимых рамках, а один брокер обслуживает много разных партиций с разным состоянием репликации.

При отказе лидера выбирают другую реплику, которая может безопасно продолжить работу. Исторически для этого использовали прежде всего ISR. В Kafka 4.x появился механизм ELR, Eligible Leader Replicas. Он также учитывает реплики вне текущего ISR, которые по состоянию журнала безопасно сделать лидерами. В новых кластерах начиная с 4.1 он включен по умолчанию.

Три копии на дисках одной физической ноды плохо защищают от его потери. Важны размещение по разным зонам отказа, настройка размещения копий по разным стойкам или зонам отказа, параметры подтверждения и сохранность самих дисков. Репликация не заменяет защиту от ошибочного удаления данных и не является архивной резервной копией.

2.7 Для чего параметр min.insync.replicas и как он связан с надёжностью?

Пусть у партиции три копии. Если две отстали или недоступны, в ISR может остаться только лидер. Тогда подтверждение всех текущих ISR само по себе еще не означает наличие нескольких надежных копий. Ограничение `min.insync.replicas` не дает незаметно снизить требования до единственной копии.

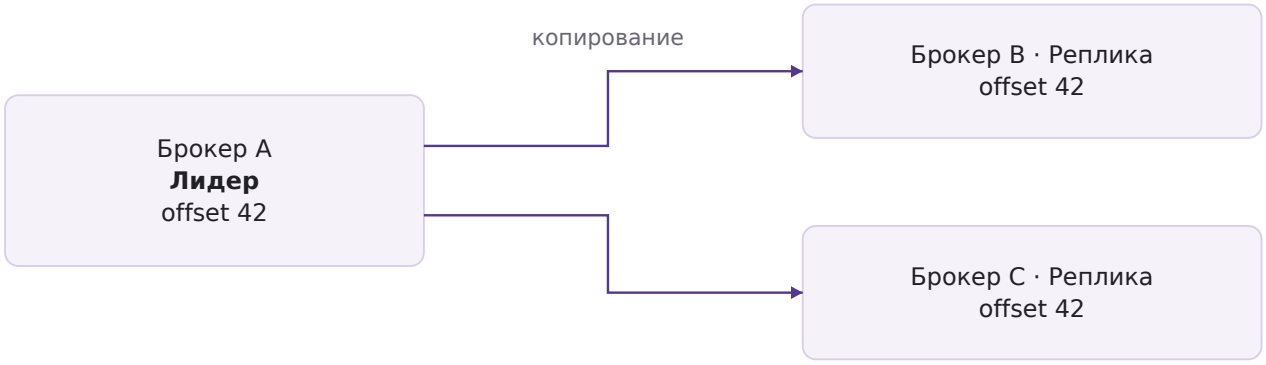
min.insync.replicas задает минимально допустимое число синхронных реплик для надежной записи с `acks=all`**.** В число входит лидер. При `min.insync.replicas=2` и ISR из одного участника продюсер получит ошибку, а не успешное подтверждение на единственной копии.

Это нижняя граница, а не количество ответов, которое всегда достаточно дождаться. Если ISR состоит из трех реплик, `acks=all` ожидает репликацию по всему текущему ISR. Значение 2 не означает «можно игнорировать третью синхронную реплику».

RF	ISR	min ISR	Результат с acks=all
3	3	2	Запись подтверждается после выполнения условий репликации
3	2	2	Запись возможна, запас по отказам уменьшился
3	1	2	Успешного подтверждения нет

Ошибка не всегда доказывает отсутствие записи. Например, состав ISR мог измениться после добавления записи в журнал. Поэтому повтор требует идемпотентного продюсера, а окончательный бизнес-результат иногда приходится уточнять.

В Kafka 4.x с ELR применяется строгая граница min ISR для продвижения high watermark, то есть границы доступных подтвержденных данных. Даже слабое `acks` не следует понимать как разрешение потребителю читать любые еще не подтвержденные записи. Настройки ELR и min ISR проверяют вместе по версии кластера.



ISR содержит A, B и C

При min ISR = 2 и acks=all учитывается весь текущий ISR

2.8 Как acks=0/1/all влияет на доставку и производительность?

Параметр acks определяет, какого подтверждения записи ждет продюсер от брокера. Он не означает, что консьюмер уже обработал запись или успешно вызвал платежный API.

Значение	Когда продюсер считает отправку подтвержденной	Риск
0	Не ожидает ответа брокера	Приложение может не узнать о потере записи
1	Лидер записал данные в свой локальный журнал	Отказ лидера до репликации способен потерять подтвержденную запись
all	Выполнены условия подтверждения текущим ISR и min ISR	Выше надежность, но возможны ожидание и отказ при недостатке реплик

Запись в локальный журнал не означает отдельный физический `fsync` для каждого сообщения. Надежность Kafka в основном строится на репликации и выбранной конфигурации. Одновременная потеря всех актуальных копий остается существенным сценарием отказа.

Чем меньше требуется подтверждений, тем меньше обязательного ожидания, но реальную задержку и пропускную способность измеряют. Размер пакетов, сжатие данных, диски и сеть могут влиять сильнее одной настройки acks. Обещание конкретного ускорения без нагрузки и измерений некорректно.

В Java-клиенте продюсера 4.1 значение по умолчанию равно all. Идемпотентность требует all, положительного числа повторов в retries и `max.in.flight.requests.per.connection` не больше 5. Если явно включить идемпотентность с несовместимыми параметрами, конфигурация завершится ошибкой.

2.9 Как работают ретраи и идемпотентный продюсер? (что это разобрать по отдельности)

Ретраи

Брокер записал пакет сообщений, но ответ потерялся. Продюсер не может понять, сохранились ли сообщения, поэтому отправляет пакет повторно. Такую попытку называют ретраем. Она помогает пережить временный сбой сети или смену лидера. Некорректные данные от повторения не исправятся.

Число попыток ограничивает retries, а общий допустимое время доставки `delivery.timeout.ms`. Настройка `request.timeout.ms` относится к ожиданию ответа на запрос. Бесконечное повторение на уровне приложения способно пережить все эти ограничения и создать новую проблему.

Идемпотентный продюсер

При `enable.idempotence=true` продюсер передает свой идентификатор и порядковый номер пакета для каждой партиции. Также используется epoch, номер поколения продюсера. Он помогает отличить прежнего отправителя от нового. По этим данным брокер распознает повтор той же отправки и не добавляет вторую копию пакета.

Это защищает от дубликатов при внутренних повторах продюсера. Но два отдельных вызова `send()` с одинаковым JSON считаются двумя отправками. Обе записи могут попасть в журнал. Брокер не понимает, относятся ли они к одному платежу.

В kafka-clients 4.1 идемпотентность включена по умолчанию при отсутствии конфликтующих настроек. Для понятного намерения ее можно задавать явно вместе с совместимыми параметрами.

PROPERTIES

```
enable.idempotence=true
acks=all
max.in.flight.requests.per.connection=5
delivery.timeout.ms=120000
```

Это фрагмент конфигурации продюсера. Адреса брокеров, преобразование объектов в байты и защиту соединения задают отдельно. Обычная идемпотентность продюсера не дает бессрочную защиту от повторения одной бизнес-команды после перезапуска приложения. Для нее нужен отдельный идентификатор операции.

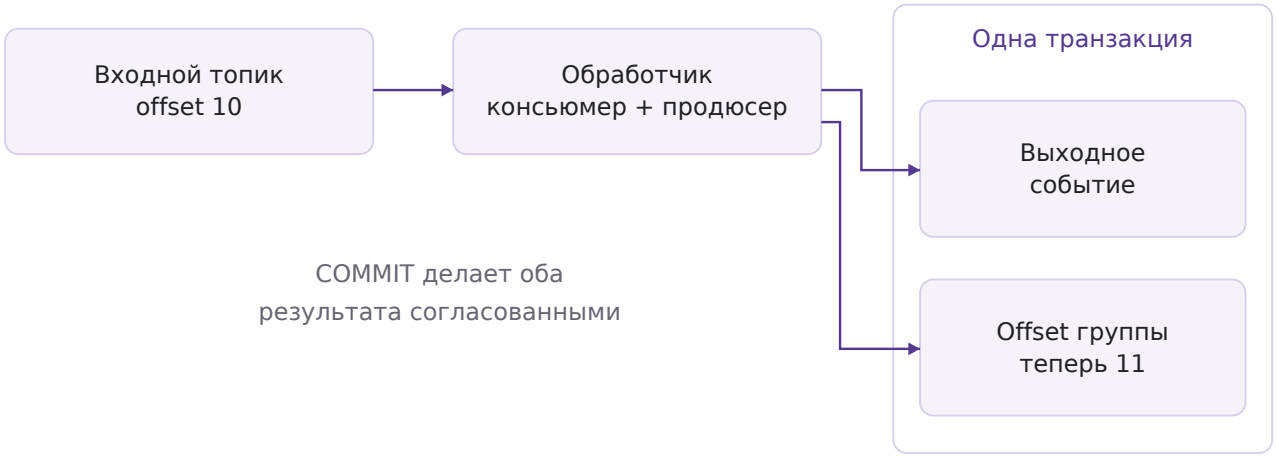
2.10 Что такое транзакции в Kafka и как ими добиться exactly-once?

Транзакция Kafka объединяет запись результатов в несколько партиций и подтверждение позиций чтения одной группы. Это полезно для цепочки «прочитать событие, вычислить результат, записать новое событие». Либо результат и продвижение позиции фиксируются вместе, либо ни одно из изменений не считается завершенным.

Пусть обработчик прочитал заказ с офсетом 10, создал событие резервирования и должен продолжить с 11. Если записать событие, а позицию подтвердить отдельно, сбой между действиями приведет к повтору результата. Если подтвердить позицию раньше записи результата, результат можно потерять.

В Kafka оба изменения можно зафиксировать одной транзакцией. Продюсер с `transactional.id` начинает ее и отправляет результат. Затем через `sendOffsetsToTransaction` добавляет новую позицию чтения и сведения о текущем участии консьюмера в группе. После этого подтверждает транзакцию. Автоматическое подтверждение позиции отключают. При отмене транзакции приложение возвращает чтение к корректной позиции перед повторной обработкой.

Получатель результатов использует `isolation.level=read_committed`. Тогда он не видит записи отмененных транзакций и не проходит границу незавершенных. Это может увеличить видимое отставание при долгой открытой транзакции.



Downstream читает read_committed. Внешняя БД вне этой границы.

Координатор транзакций хранит их состояние. Номер поколения, epoch, помогает запретить прежнему продюсеру продолжать запись после запуска нового с тем же `transactional.id`. Одновременно работающим экземплярам нужны разные идентификаторы. При восстановлении конкретного экземпляра его идентичность должна сохраняться по правилам выбранной реализации.

Идемпотентный продюсер и транзакции появились в Kafka 0.11. В Kafka Streams настройка `processing.guarantee=exactly_once_v2` управляет связанными гарантиями обработки. Но **запись в PostgreSQL, отправка письма или вызов платежного HTTP API не входят в транзакцию Kafka автоматически**. Для них нужны

собственная атомарность, идемпотентность или согласованный протокол интеграции.

2.11 acks и семантики доставки это одно и то же (acks all == exactly-once)?

Нет. Acks относится к подтверждению записи брокерами. Семантика доставки описывает, сколько раз обработка или ее результат могут произойти при сбоях. Эти вопросы связаны, но одна настройка не определяет поведение всей системы.

При обработке не более одного раза, at-most-once, допустимо потерять обработку, но не повторять ее. Например, консьюмер подтвердил позицию чтения и упал до изменения базы. После восстановления запись будет пропущена.

При обработке хотя бы один раз, at-least-once, обработку повторяют до подтвержденного результата. Консьюмер изменил базу и упал до подтверждения позиции чтения. После восстановления он получит запись снова. Это нормально для доставки, но бизнес-эффект должен выдержать повтор.

Обработка ровно один раз, exactly-once, требует определить границу гарантии. В транзакционной цепочке Kafka входные позиции чтения и выходные записи фиксируются совместно, а следующий обработчик читает только подтвержденные данные. В связке Kafka и PostgreSQL одной настройки acks для такой атомарности нет.

Представим продюсер с `acks=all`. Запись надежно сохранилась, консьюмер дважды вызвал внешнее списание после сбоя. Acks свою задачу выполнил, но деньги могли списаться дважды. Поэтому полезно отдельно обсуждать надежность хранения, повтор доставки и единственность бизнес-эффекта.

2.12 Когда выбирать compaction, а когда delete-retention, и почему?

Удаление по сроку или объему хранения, delete-retention, убирает старые сегменты журнала. Сегмент является файлом с частью последовательности записей. Такой режим подходит, если нужна история за ограниченный период. Чтение сообщений и подтверждение позиции не являются условием их удаления.

Уплотнение журнала, compaction, постепенно убирает старые значения одного ключа, сохраняя возможность восстановить последнее состояние ключа. Для потока настроек события `client-7=A`, затем `client-7=B` могут со временем превратиться в последнее значение B.

Уплотнение выполняется фоново и не является мгновенной дедупликацией. Консьюмер вправе увидеть несколько значений одного ключа. Порядок оставшихся записей и их офсеты сохраняется, а между офсетами появляются пропуски.

Для удаления состояния отправляют специальную запись с ключом и значением null. Ее называют tombstone. Это указание, что состояние данного ключа нужно удалить. Ее тоже не хранят бесконечно. Консьюмер, который строит состояние из истории,

должен успеть прочитать удаления в пределах соответствующих условий хранения, иначе в его старой копии могут остаться уже удаленные данные.

Уплотнение и удаление по сроку можно совмещать через политики compact и delete. Тогда последняя запись ключа тоже может удалиться после истечения срока хранения. Такой топик нельзя безусловно считать вечным снимком всех сущностей.

Если нужны все изменения для аудита, одного уплотнения недостаточно. Если нужно последнее состояние каждого ключа для восстановления проекции, обычное удаление по времени может стереть давно не менявшуюся сущность. Политику выбирают по задаче чтения и восстановления.

2.13 Что такое DLQ, для чего это использовать?

DLQ является отдельным местом для сообщений, которые обработчик не смог выполнить по принятой политике. В Kafka это обычный топик со своим контрактом, доступом и хранением. Он не появляется автоматически у любого консьюмера.

Временную сетевую ошибку полезно повторить с задержкой. Неподдерживаемая версия события или некорректная сумма не исправятся от тысячи мгновенных попыток. После ограниченного числа попыток такую запись можно изолировать, чтобы она не блокировала поток бесконечно.

В DLQ сохраняют исходные байты или доступную форму события, название топика, номер партиции и офсет, ключ, причину отказа, время и число попыток. Это особенно важно при ошибке десериализации, когда привычного Java-объекта еще нет. Чувствительные данные защищают так же, как в основном потоке.

Между публикацией в DLQ и commit входного offset тоже есть окно сбоя. Можно получить дубль или потерю. Если и входное сообщение, и DLQ находятся в Kafka, то эти действия при подходящей реализации объединяют транзакцией. Иначе выбирают порядок с повторной доставкой и дедупликацией.

DLQ требует владельца и процедуры повторного запуска. После исправления причины сообщения возвращают контролируемо, с прежними бизнес-ключами. Иначе DLQ превращается в незаметную потерю работы. Если последующие события обгоняют изолированное, отдельно решают проблему порядка по сущности.

2.14 Какие метрики Kafka важно мониторить в проде?

Мониторинг Kafka должен показывать сохранность данных, возможность записывать и скорость выполнения бизнес-работы. Одного графика CPU недостаточно.

На брокерах смотрят партиции без доступного лидера, партиции с неполным набором синхронных копий, число партиций ниже min ISR, изменения ISR, ошибки запросов и длительность записи сообщений и получения сообщений. Параллельно наблюдают дисковое место, задержку дисков, сеть и состояние группы контроллеров KRaft, которые совместно управляют метаданными кластера.

У продюсера смотрят ошибки и повторы отправки, время ответа брокера, размер пакетов, степень сжатия и ожидание свободного места в буфере. Буфер является областью памяти с еще не отправленными данными. Если он заполнен, приложение

может тормозить еще до обращения к брокеру. Также учитывают превышение общего времени доставки сообщения.

У консьюмера смотрят отставание по каждой партиции, скорость обработки, возраст самого старого необработанного события, длительность получения записей, ошибки подтверждения позиций и частоту ребалансировок. Отставание обычно называют lag. Среднее по группе легко скрывает одну перегруженную партицию.

Отставание можно считать по-разному. Разница между концом журнала и подтвержденной позицией не всегда равна числу невыполненных бизнес-операций. Например, приложение может подтверждать позиции редко или передавать работу в отдельные потоки. Для пользовательской задачи полезно отдельно измерять время от создания события до завершения его обработки.

Порог выбирают по допустимому времени восстановления и сроку хранения сообщений. Отставание на десять тысяч маленьких событий и на десять тысяч тяжелых платежей имеет разный смысл. Алерт должен объяснять, какие операции рискуют не выполниться вовремя.

2.15 Как контролировать размер сообщений, компрессию и батчирование (linger, batch.size)?

Продюсер собирает записи одной партиции в пакеты и отправляет их вместе. Это сокращает число сетевых обращений и обычно улучшает сжатие. `batch.size` задает ориентир размера пакета в байтах, а `linger.ms` разрешает немного подождать, чтобы собрать дополнительные записи.

Если пакет заполнился, ждать весь интервал `linger.ms` не требуется. Если поток редкий, ожидание добавляется к задержке. Реальная доставка способна занять дольше из-за очереди, сети или потому, что брокер не успевает принимать данные. `linger.ms` не ограничивает общее время доставки сообщения.

В Kafka 4.0 значение `linger.ms` по умолчанию изменилось с 0 на 5 мс. Поэтому старое утверждение «продюсер по умолчанию совсем не ждет накопления пакета» больше нельзя переносить на актуальный Java-клиент.

PROPERTIES

```
compression.type=zstd
batch.size=65536
linger.ms=5
max.request.size=1048576
```

Это учебная исходная точка для kafka-clients 4.1, а не универсальный тюнинг. Zstd экономит сеть и диски ценой CPU. На конкретных данных сравнивают размер, задержку и загрузку процессора.

Крупная запись может быть больше `batch.size`. Ограничения проверяют отдельно у продюсера, топика или брокера и консьюмера. Важны `max.request.size`, `max.message.bytes`, ограничения размера данных при копировании реплик и чтении консьюмером. Часть ограничений относится к пакету до сжатия, часть после, поэтому просто выставить везде одинаковое число недостаточно.

Большие документы и файлы обычно выгоднее хранить отдельно, а в Kafka передавать ссылку, контрольную сумму и метаданные. Но жизненный цикл внешнего объекта должен соответствовать возможности повторно читать событие.

2.16 Что такое KRaft и чем он заменил ZooKeeper?

Kafka хранит не только пользовательские сообщения, но и метаданные кластера. Нужно знать, какие существуют топики, где находятся реплики, кто является лидером партиции и какие брокеры доступны.

Раньше для части этой координации Kafka зависела от отдельного группы серверов ZooKeeper. Это добавляло еще одну систему, которую требовалось развертывать, обновлять и восстанавливать.

KRaft переносит управление метаданными в собственную группу контроллеров Kafka, которые согласуют решения по алгоритму Raft. Контроллеры согласуют журнал метаданных, а брокеры получают из него состояние кластера. Содержимое пользовательских топиков не переносится в один общий Raft-журнал. Их данные продолжают реплицироваться своим механизмом.

Чтобы согласовывать изменения, нужна доступность большинства голосующих контроллеров. Поэтому важны не только число контроллеров, но и их размещение по зонам отказа. Потеря большинства отличается от потери одного брокера с пользовательской партицией.

KRaft стал готовым для промышленной эксплуатации в линейке 3.x. Начиная с Kafka 4.0 режим ZooKeeper удален. Обновление старого ZooKeeper-кластера требует миграционного пути через поддерживаемые версии 3.x. Нельзя просто установить Kafka 4 поверх прежней конфигурации.

В промышленном кластере роли контроллера и брокера часто разводят для изоляции нагрузки и отказов. Конкретная топология зависит от масштаба, но управление метаданными всегда остается отдельной ответственностью.

2.17 Как подходить к эволюции схем (Avro/Protobuf/JSON Schema) и режимам совместимости? И нужно ли? Если да, то когда?

Событие живет дольше одной версии приложения. Новый консьюмер может читать записи, созданные месяц назад, а старый консьюмер еще работать во время нового релиза продюсера. Поэтому схема нужна не только для красивой документации.

Реестр схем, Schema Registry, хранит версии описаний сообщений и проверяет допустимость изменений. Это отдельный компонент, а не встроенная часть каждого брокера Kafka. Сообщение часто содержит идентификатор схемы. Сериализатор использует ее при превращении объекта в байты, а десериализатор при восстановлении объекта из байтов.

Режим	Что должно оставаться возможно
Backward	Новый обработчик читает прежние данные
Forward	Старый обработчик читает новые данные
Full	Оба направления
Transitive	Проверка распространяется на всю соответствующую историю версий

Обычный режим без проверки всей истории обычно сравнивает с последней схемой, поэтому для долгой истории одной такой проверки может оказаться недостаточно.

В Avro добавление поля для чтения старых записей требует подходящего значения по умолчанию в новой схеме чтения. В Protobuf нельзя переиспользовать номер удаленного поля. Его резервируют, а для JSON-представления учитывают еще имена полей и правила парсинга. В JSON Schema совместимость зависит, среди прочего, от параметров required, обязательных полей, и additionalProperties, допустимости дополнительных полей. Универсальное правило «добавление поля всегда безопасно» не работает.

Проверка формы не доказывает совместимость смысла. Замена рублей на копейки останется проблемой даже при прежнем числовом типе. Поэтому к реестру схем добавляют контрактные проверки и примеры старых сообщений.

Registry особенно полезен при нескольких независимых командах, долгом хранении и повторном проигрывании истории. В маленьком проекте можно начать с версионируемого контракта и автоматических проверок при автоматической проверке изменений перед выпуском, но правила эволюции все равно нужны.

2.18 Как реализовать «в разумных пределах» идемпотентность потребителя? И нужно ли?

Консьюмер может повторно получить сообщение после сбоя, повторного чтения истории или смены владельца партиции. Поэтому операция, которая создает платеж, отправляет деньги или увеличивает счетчик, обычно нуждается в защите от повторного эффекта.

Самый понятный вариант для PostgreSQL использует таблицу обработанных событий. Уникальный ключ включает обработчик и eventId. Запись о событии и бизнес-изменение выполняются **в одной транзакции**.

POSTGRESQL

```
BEGIN;
INSERT INTO processed_event (handler, event_id)
VALUES ('payment-projection', :eventId)
ON CONFLICT DO NOTHING
RETURNING event_id;
```

Если INSERT вернул строку, приложение применяет изменение в этой же транзакции и выполняет COMMIT. Если строк нет, событие уже обработано, изменение повторять не нужно. На `(handler, event_id)` требуется PRIMARY KEY или UNIQUE. Позицию чтения подтверждают после успешной транзакции. SQL здесь показывает первый шаг протокола, а ветвление выполняет приложение.

При сбое после COMMIT и до подтверждения позиции чтения Kafka повторит запись, но уникальный ключ не даст повторить эффект. Если бизнес-изменение откатилось, запись о факте обработки тоже откатится.

EventId должен сохраняться при транспортных повторах. Офсет защищает только конкретную запись конкретной партиции. Если продюсер заново опубликовал ту же бизнес-команду с новым офсетом, нужен бизнес-ключ операции.

Хранить идентификаторы вечно не всегда разумно. Период защиты выбирают по возможному повторному чтению истории и требованиям бизнеса. Для внешнего HTTP-эффекта локальная таблица входящих сообщений, Inbox, не дает атомарности, потребуется идемпотентность внешней стороны или сверка.

2.19 Какие типичные проблемы при высоком трафике и как их диагностировать?

При росте нагрузки первым признаком часто становится отставание консьюмера. Нужно сравнить, сколько записей приходит, сколько читается и сколько действительно обработано по каждой партиции. Само число непрочитанных сообщений еще не указывает причину задержки.

Если перегружена одна партиция, проверяют распределение ключей. Один крупный клиент может отправлять почти весь поток с одним ключом. Добавление консьюмеров не распараллелит одну партицию внутри обычной группы. Изменение ключа возможно только с учетом требований к порядку.

Если консьюмер мало использует процессор, но медленно обрабатывает, ищут ожидание базы, сети или соединений. Если обработка пакета длится дольше `max.poll.interval.ms`, возникают повторные ребалансировки и становится еще медленнее. Уменьшение `max.poll.records` или отдельная управляемая обработка могут помочь, но требуют корректного управления подтверждением позиций и владением партициями.

На стороне брокера проверяют диски, сеть, очередь запросов и состояние реплик. У продюсера смотрят заполнение буфера, размер отправляемых пакетов и ошибки. Тысячи маленьких пакетов создают лишнюю работу, а слишком большие сообщения вызывают всплески расхода памяти и задержек.

Увеличение числа партиций способно помочь только при реальном ограничении параллелизма. Оно также увеличивает метаданные и работу группы. Изменения делают по измеренной гипотезе и проверяют, не перенеслось ли узкое место в PostgreSQL или внешний API.

2.20 Когда лучше вовсе не использовать Kafka и чем её заменить?

Kafka оправдана, когда нужны поток событий, независимые потребители, хранение истории, повторное чтение истории или потоковая обработка. Если задача состоит в одном коротком запросе с немедленным ответом, HTTP или gRPC обычно проще.

Задача	Возможный выбор	Почему
Получить текущий баланс	HTTP или gRPC	Ответ нужен в рамках текущей операции
Распределять отдельные фоновые задания	RabbitMQ или управляемая очередь	Удобны подтверждение задания, маршрутизация и повтор
Надежно выполнить небольшую фоновую работу рядом с БД	Таблица заданий и фоновый обработчик	Меньше инфраструктуры, прием задания можно связать с транзакцией
Раздать историю событий нескольким группам	Kafka	Независимые позиции чтения и повторное проигрывание
Короткие некритичные уведомления	Публикация и подписка подходящего сервиса	Допустима потеря, не нужен долговременный журнал

Сравнение не означает, что RabbitMQ не умеет хранить данные или Kafka не умеет выполнять задания. Отличаются модель работы и цена нужных гарантий. Например, удерживать долгое индивидуальное задание и независимо подтверждать его в Kafka требует дополнительного проектирования при использовании обычной группы консьюмеров.

Для небольшой команды затраты на кластер, схемы сообщений, права доступа, хранение и эксплуатацию могут превысить пользу. Нельзя выбирать Kafka только потому, что приложение называется микросервисным. Сначала определяют требования к доставке, порядку, задержке и восстановлению, затем выбирают транспорт.

3. Kubernetes

3.1 Зачем Kubernetes и какие задачи он решает для микросервисов?

Приложение уже собрано в контейнерный образ. Теперь нужно запускать несколько экземпляров, размещать их на серверах, обновлять, подключать сеть и восстанавливать после сбоев. Kubernetes управляет этой работой через описание желаемого состояния.

Мы сообщаем Kubernetes, что хотим три экземпляра сервиса определенной версии. Он запускает их в Pod. Pod объединяет контейнеры, которые должны работать вместе. Если один Pod исчез, контроллер создает замену. Контроллер является частью Kubernetes, которая постоянно сравнивает нужное состояние с фактическим и исправляет расхождения.

Нода является сервером или виртуальной машиной, на которой запускаются Pod. Управляющая часть кластера принимает команды через API server и хранит сведения о состоянии в etcd. Планировщик, scheduler, выбирает ноду для нового Pod. Агент kubelet на этой ноде следит за контейнерами и обращается к программе, которая их запускает, например containerd. Сетевые компоненты и драйверы хранилищ подключают сеть и диски.

Kubernetes управляет выполнением приложения, но не понимает его бизнес-правил. Он не знает, можно ли повторить списание, завершена ли Saga и совместимы ли события двух версий. Это остается ответственностью приложения и команды.

Платформа полезна, когда требуется единообразно эксплуатировать множество компонентов. Для маленького приложения простой контейнерный хост или управляемая платформа могут быть дешевле. Микросервисы можно запускать без Kubernetes, а монолит можно запускать в Kubernetes.

3.2 Что такое Pod и почему это минимальная единица деплоя?

Pod объединяет один или несколько контейнеров, которые нужно запускать вместе на одной ноде. У них общая сеть, поэтому один контейнер может обратиться к другому через localhost, как будто они находятся на одном компьютере. Также им можно подключить общий том, то есть место для хранения файлов.

Например, основной контейнер запускает Java-приложение, а вспомогательный выполняет тесно связанную работу. Но база данных и все клиенты не должны попадать в один Pod только потому, что им нужно общаться.

Kubernetes размещает, адресует и заменяет Pod как единицу. Нельзя увеличить число только одного контейнера внутри уже существующего Pod как число

независимых экземпляров приложения. Для такого масштабирования создают дополнительные Pod.

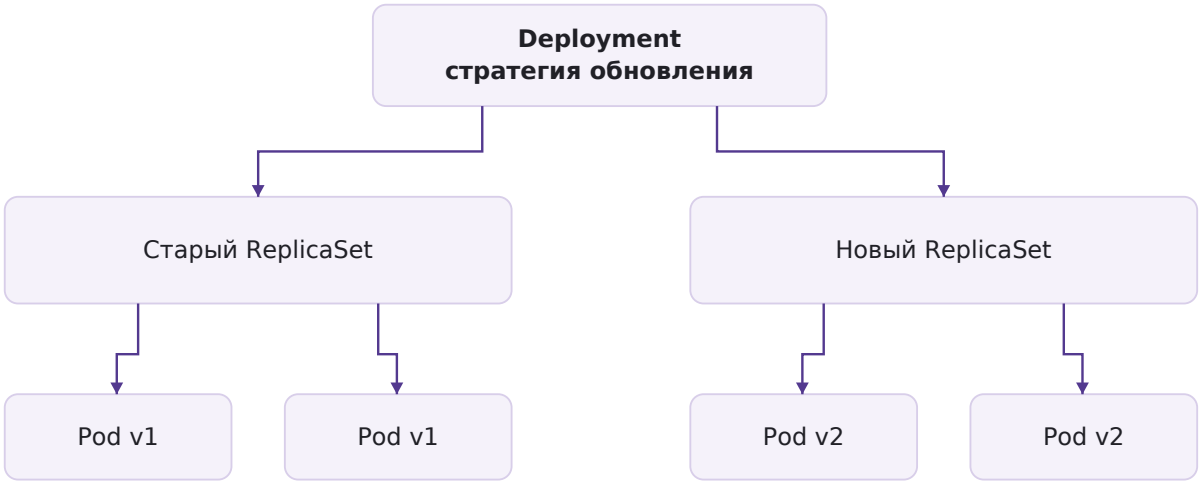
У каждого Pod есть собственная идентичность и жизненный цикл. При потере ноды контроллер создает новый Pod, а не переселяет прежний процесс вместе с памятью. IP тоже может измениться, поэтому к сервису обычно обращаются через Service, который дает постоянную точку обращения к подходящим Pod. DNS позволяет найти ее по имени, например orders.

Общий сетевой контекст не означает полностью общую файловую систему. Корневые файловые системы контейнеров отдельные. Для обмена файлами явно монтируют том. Общее пространство процессов также не следует считать включенным автоматически.

3.3 Для чего нужны Deployment и ReplicaSet, чем отличаются?

ReplicaSet поддерживает нужное число Pod. Он находит их по меткам, labels. Например, метка `app=orders` обозначает Pod сервиса заказов, а selector задает правило «выбрать Pod с этой меткой». Если нужны три реплики, но осталось две, контроллер создает недостающую. Порядком выпуска новой версии управляет уже Deployment.

Deployment управляет ReplicaSet и историей изменений шаблона Pod. При релизе он создает новый ReplicaSet, постепенно увеличивает число новых Pod и уменьшает старые согласно стратегии обновления.



Deployment меняет число Pod через управляемые ReplicaSet

Поэтому в обычном приложении создают Deployment, а не вручную редактируют управляемые им ReplicaSet и Pod. Иначе ручное изменение может быть перезаписано контроллером или конфликтовать с желаемым состоянием.

ReplicaSet считает экземпляры по меткам, поэтому правила выбора Pod должны быть точными и не пересекаться случайно. Новая версия шаблона Pod дает новую ревизию Deployment, а изменение только числа реплик не является новой версией кода.

Откат версии, rollback, возвращает прежний шаблон Pod. Он не откатывает автоматически миграции PostgreSQL, внешние платежи и измененные данные. План безопасного релиза должен учитывать эти действия отдельно.

3.4 Какие типы Service (ClusterIP/NodePort/LoadBalancer) и когда их использовать?

Pod может исчезнуть и получить замену с другим IP. Service дает постоянный адрес для обращения к группе подходящих Pod. Он выбирает их по меткам. Ресурс ЭндпоинтSlice хранит конкретные адреса и сведения о том, какие Pod готовы принимать запросы. Поэтому клиенту не нужно заново искать каждый экземпляр приложения.

Тип	Назначение	Типичная ситуация
ClusterIP	Виртуальный адрес внутри кластера	Вызовы между внутренними сервисами
NodePort	Выделяет порт на нодах в дополнение к Service	Внешний балансировщик или специальный доступ через ноды
LoadBalancer	Запрашивает внешний балансировщик у интеграции платформы	Публикация точки входа наружу

LoadBalancer требует отдельной реализации. В облаке это обычно контроллер, который заказывает внешний балансировщик у облачного провайдера. Балансировщик распределяет входящие соединения между доступными экземплярами. Один объект Service не создает внешний балансировщик без такой интеграции. Способ доставки трафика до Pod зависит от реализации.

ClusterIP не означает публичный HTTP API, а Service обычно не анализирует путь HTTP-запроса. Балансировка выполняется сетевым механизмом платформы. Это может быть kube-proxy или другая реализация, например на eBPF.

Для стабильных адресов отдельных экземпляров используют Service без общего виртуального IP, headless Service, с `clusterIP: None`. У него нет обычного виртуального ClusterIP, а DNS помогает находить адреса отдельных Pod. Это часто нужно StatefulSet, где важно различать участников.

3.5 Что такое ConfigMap и Secret, чем они различаются по назначению?

ConfigMap хранит несекретные настройки. Например, адрес внешнего API, название топика и флаг включения функции. Secret предназначен для чувствительных значений, например пароля или токена.

Оба объекта можно передать через переменные окружения или смонтировать в файлы. Переменная окружения уже работающего процесса не меняется после

обновления объекта. Файл обычного тома обновляется с задержкой, но приложение должно уметь перечитать его. Монтирование через subPath не дает такого автоматического обновления.

Base64 в поле data объекта Secret является кодировкой, а не шифрованием. Любой, кто получил значение, может его декодировать. По умолчанию нельзя рассчитывать, что все Secret обязательно зашифрованы в etcd. Защиту хранения настраивают отдельно.

Права на Secret ограничивают через RBAC. Это система прав Kubernetes, в которой роли разрешают конкретные действия с конкретными ресурсами. Важно ограничивать и создание Pod. Тот, кто может создать Pod в namespace, отдельной области кластера, иногда способен подключить к нему Secret из этой области. Поэтому одного запрета на команду `kubectl get secret` недостаточно.

ConfigMap не стоит использовать как базу изменяемых бизнес-данных. Secret не заменяет полноценное хранилище секретов с выдачей короткоживущих данных доступа, аудитом и ротацией. Это объекты API Kubernetes конфигурации приложения.

3.6 Что делают liveness/readiness/startup-probes и как настраивать?

Pod может иметь статус Running, но Java-приложение в его контейнере еще загружается или уже зависло. Kubernetes нужны дополнительные проверки, чтобы решить, направлять ли сюда запросы и нужен ли перезапуск. Эти проверки называются probes, или пробы. Каждая настраивается для конкретного контейнера и отвечает на свой вопрос.

Проба	Что проверяет	Последствие устойчивой неудачи
Startup	Завершился ли запуск контейнера	Перезапуск контейнера по его политике
Liveness	Способен ли процесс продолжать полезную работу	Перезапуск контейнера по его политике
Readiness	Можно ли направлять сюда обычные запросы	Обычные запросы через Service перестают направляться в этот Pod

Startup probe проверяет, запустилось ли приложение. Пока она не успешна, Kubernetes не выполняет liveness и readiness. Liveness проверяет, не зависло ли приложение так, что ему нужен перезапуск. Readiness проверяет, можно ли сейчас принимать запросы. Например, во время загрузки кэша приложение может быть живым, но еще не готовым обслуживать пользователей.

YAML

```yaml
# Фрагмент spec.containers[] для HTTP-порта 8080
ports:
  - name: http
    containerPort: 8080
startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: http
  periodSeconds: 5
  failureThreshold: 24
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: http
  periodSeconds: 10
  timeoutSeconds: 2
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: http
  periodSeconds: 5
  timeoutSeconds: 2
```

В примере используется Spring Boot Actuator, библиотека с готовыми служебными эндпоинтами. Для работы нужны включенные проверки состояния и доступ к указанным путям. Startup probe допускает примерно две минуты запуска. Это учебные значения, реальные интервалы выбирают по тому, сколько приложение запускается на практике.

Liveness не должна падать только потому, что общая база временно недоступна. Иначе все экземпляры начнут перезапускаться одновременно. В readiness зависимости включают осознанно. Если все Pod исключаются при сбое одной общей базы, трафику будет некуда идти. Если служебные проверки вынесены на отдельный порт, дополнительно проверяют, что жив именно рабочий HTTP-сервер.

3.7 Что такое requests/limits, зачем они нужны и как влияют на HPA?

Request задает количество ресурсов, которое Kubernetes учитывает при выборе ноды для Pod. Например, контейнер запрашивает `500m` CPU и `512Mi` памяти. Значение `500m` соответствует половине одного процессорного ядра по доступному времени. Планировщик ищет ноду, где с учетом запросов остальных Pod есть место.

Limit ограничивает расход ресурсов контейнером. При достижении лимита CPU приложение обычно начинает ждать следующую порцию процессорного времени. Это называют throttling. С памятью иначе. Если контейнеру требуется больше разрешенного, ядро может завершить его из-за нехватки памяти. В Kubernetes такая причина видна как OOMKilled.

```yaml
resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: "2"
    memory: 1Gi
```

Это фрагмент описания контейнера. На Linux с соответствующими cgroups механизмы применяет runtime и ядро. Requests не запрещают использовать больше при наличии ресурсов, а ограничения по памяти не следует понимать как резервирование физической RAM навсегда.

HPA автоматически меняет число Pod в зависимости от нагрузки. При использовании CPU utilization он сравнивает фактический расход процессора с request. Если приложение использует 350m при request 500m, получается 70%. При request 100m та же нагрузка выглядит как 350%. Поэтому заниженный request может привести к лишним репликам.

Без нужных requests доля использования относительно request может оказаться неопределенной. HPA по абсолютным или внешним метрикам работает иначе. Для JVM memory limit должен включать не только heap, но и память метаданных классов, буферы вне кучи Java, стеки потоков и память процесса. `-Xmx` равный всему limit оставляет мало места остальному.

3.8 Для чего нам вообще пробы?

Pod запущен, но его приложение может не отвечать на запросы. Например, все рабочие потоки зависли в ожидании, соединения перестали приниматься или приложение еще загружает данные. Один статус Running этого не показывает.

Пробы нужны, чтобы Kubernetes мог различать эти ситуации. Readiness убирает неготовый Pod из обычного трафика Service. Liveness позволяет перезапустить контейнер, если приложение зависло. Startup дает приложению время запуститься, прежде чем включатся остальные проверки. Неудачная liveness-проверка обычно перезапускает контейнер внутри Pod, а не пересоздает весь Pod.

Представим, что Pod уже создан, но приложение внутри еще загружает кэш. Проверка readiness возвращает неготовность, и обычные запросы идут в другие Pod. Когда кэш загружен, проверка начинает проходить и этот Pod тоже принимает запросы.

Однако проба является только наблюдением конкретного условия. Ответ 200 от пустого эндпоинт не доказывает, что можно оформить заказ. Слишком тяжелая проверка сама создает нагрузку, а слишком чувствительная вызывает колебания готовности.

Пробы помогают платформе управлять жизненным циклом, мониторинг помогает людям понимать качество сервиса. Это связанные, но разные задачи.

Сервис может быть готовым и при этом не выполнять целевые показатели качества из-за высокой задержки или ошибок в бизнес-правилах.

3.9 В чём разница между StatefulSet, Deployment и DaemonSet?

Deployment подходит взаимозаменяемым экземплярам приложения. Любой экземпляр API способен принять запрос, а долгоживущее состояние хранится во внешней базе или другом надежном хранилище.

StatefulSet нужен, когда экземпляры нельзя считать полностью взаимозаменяемыми. Например, у каждого участника базы свой диск и постоянное имя. Pod получают имена `db-0`, `db-1` и отдельные заявки на диски, PVC. PVC является запросом приложения на постоянное хранилище. В следующем вопросе разберем его связь с самим диском. При замене Pod сохраняется связь его имени с нужными данными.

DaemonSet запускает Pod на каждом подходящем ноде. Так удобно развертывать агент сбора логов, сетевой компонент или агент сбора показателей ноды. Он управляет покрытием нод, а не обычным числом взаимозаменяемых реплик API.

Контроллер	Что сохраняет	Основная задача
Deployment	Желаемое число подходящих Pod	Масштабирование и обновление приложения с взаимозаменяемыми экземплярами
StatefulSet	Идентичность и связь экземпляра с хранилищем	Управление участниками системы с состоянием отдельных экземпляров
DaemonSet	Наличие экземпляра на подходящих нодах	Инфраструктурная функция на ноде

StatefulSet не реализует репликацию базы, выбор лидера и резервное копирование. Эти механизмы должны предоставлять сама система и, при необходимости, оператор Kubernetes, специальный контроллер для управления конкретным приложением. Наличие локального кэша в Java-приложении тоже не означает, что ему обязательно нужен StatefulSet.

3.10 Как работают PV/PVC/StorageClass и когда нужен StatefulSet?

Контейнер можно заменить, а важные данные должны пережить замену. Kubernetes разделяет заявку на хранилище и само предоставленное хранилище.

PersistentVolume, PV, описывает выделенное постоянное хранилище, например подключаемый диск. PersistentVolumeClaim, PVC, является заявкой приложения на такое хранилище. В заявке указывают размер и режим доступа. StorageClass задает, как создавать диски по заявкам, например какой тип диска использовать и каким драйвером его подключать.

Pod ссылается на PVC, а контроллер и драйвер хранилища выделяют и подключают нужный диск. Такие драйверы работают через интерфейс CSI. Если диск доступен только в определенной зоне, Pod тоже должен находиться в подходящей зоне. Режим `WaitForFirstConsumer` откладывает выделение хранилища до выбора места для первого Pod, чтобы эти решения не противоречили друг другу.

ReadWriteOnce означает возможность монтирования для записи одной нодой, а не обязательно одним Pod. Для ограничения одним Pod существует ReadWriteOncePod при поддержке нужного CSI. Реальные гарантии и режимы зависят от драйвера хранилища.

В StatefulSet `volumeClaimTemplates` создает отдельные PVC для экземпляров. Это удобно для базы, где у каждого участника собственный диск. Но Deployment тоже способен использовать PVC, поэтому PVC сам по себе не требует StatefulSet.

При удалении важны настройки удаления PV и сохранения PVC. Они определяют, останется ли хранилище после удаления заявки или StatefulSet. Удаление Pod, StatefulSet, PVC и физического диска являются разными действиями. Перед эксплуатацией проверяют, что именно сохранится, как восстановить данные и как подключение диска поведет себя после отказа ноды.

3.11 Что такое Ingress и чем он отличается от Gateway-подходов?

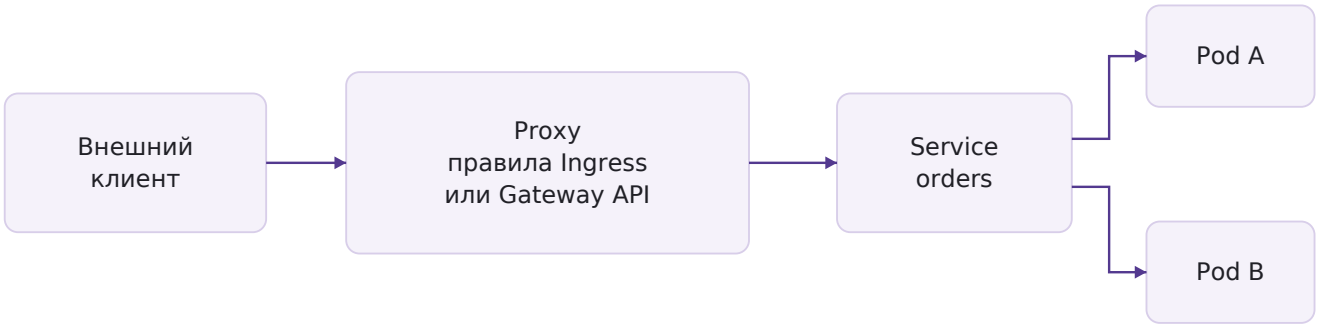
Ingress описывает правила входящего HTTP и HTTPS трафика. Например, запросы к `shop.example` с путем `/orders` нужно направить к Service заказов. Для выполнения правил нужен Ingress-контроллер. Один созданный YAML не начинает проксировать трафик.

Обычно контроллер управляет прокси-сервером или внешним балансировщиком, который принимает соединение, завершает TLS при выбранной конфигурации и направляет запрос дальше. Возможности сверх базового API часто задаются аннотациями конкретного контроллера.

Ingress API заморожен для новых возможностей. Он продолжает работать, но Kubernetes рекомендует Gateway API для развития сетевой модели.

В Gateway API обязанности разделены. GatewayClass выбирает реализацию сетевого компонента. Gateway описывает точки приема соединений, например HTTPS-порт и домен. HTTPRoute задает правила, по которым запросы передаются сервисам. Для работы устанавливают дополнительные типы ресурсов Kubernetes, CRD, и контроллер, который умеет их обслуживать. Поэтому инфраструктурная команда может управлять входом, а команда приложения своими маршрутами.

Gateway API и API Gateway являются разными понятиями. Первое является набором Kubernetes API для конфигурации сети. Второе является архитектурной ролью входной точки, которая может включать авторизацию, квоты и агрегацию. Spring Cloud Gateway может выполнять эту роль, но не становится автоматически Kubernetes контроллером Gateway API.



Логический путь. Конкретный proxy может направлять трафик прямо к endpoints.

3.12 Как работают taints/tolerations и node/pod affinity?

Affinity задает пожелания или требования к размещению Pod. Node affinity учитывает свойства нод, например тип процессора или зону. Простой вариант правила, nodeSelector, выбирает ноды по меткам. Метки являются парами ключа и значения, которыми администратор описывает ноду.

Обязательное правило affinity является жестким условием. Если подходящей ноды нет, Pod остается Pending. Предпочтительное правило affinity влияет на оценку вариантов, но не обещает размещение именно там.

Pod affinity учитывает соседство с другими Pod, а anti-affinity помогает разнести экземпляры. Для снижения общего риска реплики полезно распределять по нодам и зонам. Правила распределения по зонам, topology spread constraints, ограничивают, насколько сильно число реплик может различаться между нодами или зонами.

Taint задает ограничение на самой ноде. Он помечает нода как неподходящий для Pod без разрешения на такое ограничение, toleration. `NoSchedule` мешает новому размещению, `PreferNoSchedule` выражает мягкое ограничение, `NoExecute` может привести еще и к выселению уже работающих Pod.

Toleration разрешает размещение на помеченной ноде, но не заставляет планировщик выбрать эту ноду. Для выделенных нод часто нужны одновременно taint с toleration и affinity или selector. Иначе Pod может уйти на обычную ноду, а выделенная останется пустой.

Слишком жесткая anti-affinity способна остановить релиз, если нод меньше, чем требуется для всех старых и новых реплик. Правила размещения проверяют вместе с maxSurge и реальной емкостью кластера.

3.13 Что такое NetworkPolicy и как ограничивать сетевые потоки?

NetworkPolicy задает разрешенные сетевые взаимодействия выбранных Pod. Она работает только при наличии сетевого плагина, которая действительно поддерживает эти правила. Сам API server не фильтрует пакеты.

Без изолирующих политик Pod обычно не ограничены этим механизмом. Когда Pod выбирается политикой входящего трафика, его входящие соединения должны соответствовать разрешающим правилам. Несколько политик складываются, а не перекрывают друг друга по принципу последней записи.

Для соединения, где обе стороны изолированы, должны разрешать его исходящие правила отправителя и входящие правила получателя. Правила описывают Pod, областей namespace, IP-блоки и порты. Стандартная NetworkPolicy не является HTTP-фильтром по пользователю или URL.

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: orders-from-gateway
spec:
  podSelector:
    matchLabels:
      app: orders
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: gateway
      ports:
        - protocol: TCP
          port: 8080
```

Пример разрешает вход на orders от gateway в том же namespace на TCP 8080. Он не ограничивает исходящий трафик и не отменяет разрешения других политик. Если понадобится запрет исходящих соединений, отдельно сохраняют нужный доступ к DNS, базе и сервисам сбора метрик и логов.

Для другого namespace задают namespaceSelector, условие выбора областей, и нужное сочетание правил выбора Pod. Прежде чем включать политику с запретом всего трафика по умолчанию, проверяют все необходимые потоки. Иначе легко случайно сломать разрешение DNS или доступ к управляющим компонентам кластера.

3.14 Как проводить безопасные релизы: rolling update параметры, maxSurge/maxUnavailable?

Постепенное обновление, rolling update, заменяет старые Pod новыми. Параметр maxSurge ограничивает допустимое превышение желаемого числа, maxUnavailable ограничивает число недоступных относительно желаемого.

Для четырех реплик с `maxSurge=1` и `maxUnavailable=0` Deployment может создать пятый Pod и дождаться его доступности, прежде чем убрать старый. При нехватке CPU, памяти или подходящих нод обновление остановится, даже если конфигурация логически верна.

YAML

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
minReadySeconds: 10
progressDeadlineSeconds: 300
```

Это фрагмент spec Deployment. Готовность нового Pod должно отражать реальную готовность. Параметр minReadySeconds помогает не считать краткий успешный старт устойчивой доступностью. Параметр progressDeadlineSeconds задает срок ожидания прогресса обновления. По его истечении Kubernetes отмечает проблему, но не откатывает версию автоматически.

В процентах maxUnavailable округляется вниз, maxSurge вверх. Для одной реплики это существенно. Одновременный ноль обоих параметров не позволяет продвигать обычное постепенное обновление.

Старая и новая версии некоторое время работают вместе. Миграции базы и контракты должны это выдерживать. При остановке приложение прекращает прием новой работы и корректно завершает текущую в пределах terminationGracePeriodSeconds, времени на корректную остановку. Балансировщики и уже открытые соединения учитывают отдельно.

PodDisruptionBudget ограничивает, сколько реплик можно добровольно вывести из работы, например при обслуживании ноды через механизм выселения Pod. Он не заменяет настройки Deployment и не защищает от всех отказов. При canary-релизе сначала проверяют новую версию на небольшой доле трафика. При blue-green готовят вторую полную версию и переключают трафик на нее. Эти подходы тоже требуют правил отката и проверки результата.

3.15 Как организовать секреты: KMS, sealed-secrets, внешние хранилища?

Здесь есть несколько разных задач. Нужно защитить секрет в Git, защитить его хранение в кластере, безопасно передать приложению и менять без длительного простоя. Один инструмент редко закрывает все сразу.

KMS-интеграция помогает шифровать данные в хранилище etcd. KMS является внешней системой управления ключами. Kubernetes использует ключи для шифрования данных, а KMS защищает ключи более высокого уровня. Но приложение или пользователь с разрешенным доступом к Secret все равно получает его значение. Шифрование диска не защищает от утечки через API или лог.

Sealed Secrets позволяет сохранить в Git зашифрованный объект. Контроллер в кластере расшифровывает его и создает обычный Secret. Закрытый ключ контроллера нужно защищать и иметь план восстановления. Открытая зашифрованная форма сама по себе не является обычным Kubernetes Secret.

External Secrets Operator получает значения из внешнего хранилища и синхронизирует их в Secret. Это удобно для централизованной ротации, но означает, что копия значения существует в кластере. Через драйверы CSI секреты можно передавать в Pod как файлы, а детали хранения зависят от реализации.

Приложению дают только нужные права и по возможности короткоживущие пароли или другие данные доступа. Ротация включает обновление источника, доставку нового значения и проверку того, что клиент базы или API действительно начал его использовать. Изменение Secret не перезапускает автоматически все приложения и не заменяет уже открытые соединения.

3.16 Какие ключевые метрики и эвенты смотреть при инциденте?

При инциденте сначала определяют масштаб. Проблема у одного Pod, одной ноды, одной зоны или всех экземпляров? Затем связывают ее со временем релиза, миграции и изменения конфигурации.

Для приложения смотрят частоту ошибок, задержку запросов и число одновременно выполняемых операций. P95 и p99 показывают задержку, в которую укладываются 95% и 99% запросов. Для Java также проверяют память, работу сборщика мусора, занятость потоков, соединений с базой и время ожидания внешних сервисов.

Для Pod важны готовность, число перезапусков и причина последнего завершения контейнера. OOMKilled означает завершение из-за памяти, exit code является кодом завершения программы. На ноде проверяют процессор, память, свободное место и сеть. Состояния MemoryPressure и DiskPressure сигнализируют о нехватке памяти или дисковых ресурсов. Средние показатели по кластеру могут скрывать одну перегруженную ноду.

События Kubernetes, Events, объясняют решения платформы. FailedScheduling говорит о невозможности размещения, FailedMount о проблеме подключения volume, BackOff о задержке повторных запусков, Unhealthy о неуспешной пробе. Конкретная причина содержится в сообщении, само имя события не является диагнозом.

События хранятся ограниченное время и не заменяют долговременный аудит. Для расследования сохраняют необходимые наблюдения, сопоставляют их с логами и трассировками запросов. Перезапуск иногда временно снимает симптом, но может уничтожить полезные данные о зависании, поэтому сначала стоит понять, какую информацию необходимо успеть получить.

3.17 Как дебажить «под висит/крашится» (kubectl describe/logs, events)?

Сначала нужно понять, на каком этапе проблема. Pending означает, что Pod еще не удалось подготовить к работе. Running означает, что он уже запущен, но приложение все равно может быть неготовым. CrashLoopBackOff означает повторяющиеся падения контейнера с увеличением паузы между запусками. Это последствие ошибки, саму причину нужно найти в логах и описании Pod.

```shell
kubectl get pods -n orders -o wide
kubectl describe pod orders-abc -n orders
kubectl logs orders-abc -n orders -c app --tail=200
kubectl logs orders-abc -n orders -c app --previous
kubectl get events -n orders --sort-by=.metadata.creationTimestamp
```

В командах указаны учебные имена namespace, Pod и контейнера. Команда describe показывает состояние, предыдущую причину завершения, проверки и события. Флаг `--previous` у logs позволяет увидеть логи предыдущего запуска контейнера в том же Pod. Он полезен, если контейнер уже успел перезапуститься.

Для Pending проверяют, хватает ли ресурсов, не превышены ли квоты и подходят ли ноды по правилам размещения. Затем проверяют заявку PVC на диск. ImagePullBackOff означает, что образ контейнера не удалось получить. Здесь важны имя образа, доступность хранилища образов и права доступа. При OOMKilled сравнивают лимит памяти с расходом всей JVM, а не только кучи Java. Один код 137 еще не доказывает нехватку памяти, нужно смотреть причину завершения.

Если Pod имеет статус Running, но запросы зависают, проверяют readiness, правила выбора Pod в Service и адреса в ЭндпоинтSlice. Затем проверяют DNS и NetworkPolicy. Если сеть работает, смотрят приложение, занятость потоков и его зависимости. Ответ служебного порта еще не доказывает, что основной порт тоже работает.

В минимальном образе может не быть командной оболочки и диагностических программ. Тогда при наличии прав можно добавить временный контейнер для диагностики через kubectl debug. Для Java также полезны заранее подготовленные средства получения состояния потоков и памяти. Такие действия ограничиваются правами RBAC и политиками кластера.

3.18 Что учитывать при автоскейлинге (HPA/VPA, метрики, холодный старт)?

HPA меняет число Pod. VPA рекомендует или меняет выделяемые контейнеру ресурсы. Автомасштабирование нод добавляет или убирает сами серверы кластера. Это три разные задачи. HPA может запросить новые Pod, но они останутся Pending, если свободных ресурсов на нодах нет.

Метрику выбирают по реальному ограничению. CPU полезен для вычислительной нагрузки. Для ожидания внешнего API лучше могут работать число одновременных

запросов и длина очереди. Для Kafka учитывают отставание обработки, скорость обработки и число партиций.

Базовая идея HPA выражается отношением текущего показателя к целевому. Если при трех репликах средняя использование CPU относительно request 120%, а цель 60%, грубая оценка дает шесть реплик. Реальный алгоритм дополнительно учитывает пропуски метрик, неготовые Pod, допуски и ограничения изменения масштаба.

Масштабирование действует с задержкой. Нужно получить метрику, принять решение, выделить нода при необходимости, скачать образ и запустить JVM. Поэтому минимальное число реплик и резерв емкости важны даже при настроенном HPA.

Чтобы избежать колебаний, задают период стабилизации, в течение которого учитывают предыдущие рекомендации и ограничивают скорость роста и уменьшения. Новые экземпляры могут одновременно открыть много соединений к PostgreSQL, поэтому масштаб API проверяют вместе с лимитом базы.

VPA не следует бездумно сочетать с HPA по использование CPU относительно request. Если VPA меняет CPU request, меняется база расчета процента использования CPU. Способ применения VPA и возможность изменения без пересоздания зависят от версии и политики. Рекомендательный режим позволяет сначала оценить значения без автоматического воздействия.

3.19 Что такое Ingress, istio и че это вообще значит все?

Ingress задает правила входа HTTP-трафика в кластер. Istio управляет сетевым взаимодействием сервисов внутри системы. Такой дополнительный сетевой слой называют service mesh. Например, он может шифровать соединения между сервисами, проверять доступ и собирать сведения о вызовах без одинакового кода в каждом приложении.

У Istio есть управляющие компоненты, которые распространяют настройки, и компоненты обработки трафика, через которые идут соединения. В документации их называют control plane и data plane. Настройки определяют, куда направить запрос, какие соединения разрешить и как проверять участников. Взаимный TLS означает, что обе стороны соединения подтверждают свою личность сертификатами.

В классическом варианте рядом с приложением запускается отдельный контейнер с прокси-сервером. Его называют sidecar. Приложение отправляет обычный запрос, а прокси-сервер применяет правила передачи трафика. В другом варианте, ambient, часть этих функций вынесена из Pod в общие компоненты. Поэтому Istio не обязательно означает отдельный прокси-сервер в каждом Pod. Названия ztunnel и waypoint относятся к компонентам этого второго варианта, а не к новым видам бизнес-сервисов.

Service Kubernetes дает сетевую точку обращения, Ingress-контроллер или реализация Gateway API организует вход, а Istio может управлять внутренним обменом. Сам Kubernetes не требует Istio для обычного вызова между Pod.

Istio добавляет собственную стоимость, задержку и сложность диагностики. Он не знает, можно ли повторить платеж, и не превращает сетевой retry в безопасную бизнес-операцию. Если повтор запроса одновременно настроен в приложении, прокси-сервере и входном шлюзе, их взаимодействие необходимо согласовать.

3.20 В чем разница между k8s и Kubernetes (минимум 10 отличий)?

https://www.youtube.com/watch?v=xJQPCwOm3dk

4. REST, gRPC и Kafka

4.1 Почему не стоит всё делать только REST, и когда он уместен? Нормально ли базировать всю систему на REST?

Базировать систему на HTTP API вполне нормально, если ее требования этим покрываются. REST описывает архитектурный стиль работы с ресурсами и ограничениями взаимодействия. Он не равен любому JSON поверх HTTP, хотя в разговоре эти вещи часто смешивают.

Для внешних клиентов HTTP удобен доступностью инструментов, понятными методами и статусами, поддержкой кэширования и развитой инфраструктурой. Получить заказ, создать заявку и прочитать ее состояние можно без брокера.

Проблема возникает, когда каждый бизнес-процесс превращается в длинную цепочку синхронных запросов. Время ожидания складывается, а зависимость от доступности всех участников усиливается. Уведомления и аналитика не обязаны задерживать подтверждение уже созданного заказа.

HTTP не запрещает асинхронную модель. Сервер может надежно принять работу, вернуть 202 и предоставить ресурс операции для проверки статуса. И наоборот, наличие Kafka не избавляет от синхронных зависимостей, если отправитель все равно ждет ответное событие перед продолжением.

Поэтому выбирают способ взаимодействия для конкретной задачи. Небольшая система из нескольких HTTP-сервисов может быть проще и надежнее сложной событийной архитектуры. Добавлять другой протокол стоит тогда, когда понятна проблема, которую он решит.

4.2 В чём преимущества gRPC по сравнению с REST для внутрянки?

gRPC дает модель удаленного вызова методов, RPC. Мы описываем сервис и методы в `.proto`, после чего инструменты создают клиентские классы, которые отправляют запрос и читают ответ вместо написанного вручную HTTP-клиента и серверные интерфейсы. Вызывающий работает с типизированным методом, хотя фактически происходит сетевой обмен.

Обычно gRPC использует Protobuf для сообщений и HTTP/2 как транспорт. Бинарное представление может уменьшать объем данных, а генерация клиентов помогает поддерживать одинаковую форму контракта между разными языками.

Полезны возможность задать крайний срок завершения вызова, deadline, статусы RPC, метаданные, дополнительные поля вызова наподобие HTTP-заголовков и разные виды потоковой передачи сообщений. Для внутреннего обмена с большим количеством типизированных вызовов это часто удобнее ручного согласования JSON-клиентов.

Но **gRPC не гарантирует, что вызов всегда быстрее HTTP API**. Результат зависит от размера данных, сериализации, сети, подключения и обработки. HTTP API тоже может работать по HTTP/2 и передавать бинарное тело.

Диагностика gRPC требует подходящих инструментов и поддержки прокси-серверов. Для браузера нередко нужны gRPC-Web или специальный промежуточный слой, а не обычный прямой вызов. Публичный API для разнообразных клиентов часто удобнее оставить HTTP/JSON.

Сгенерированный Java-метод не делает вызов локальным. Таймаут, повторная доставка, частичный отказ и авторизация остаются обязательными вопросами проектирования.

4.3 Когда выбирать асинхронность через Kafka вместо синхронного RPC?

Kafka выбирают, когда работу можно отделить по времени от запроса пользователя. После создания заказа отправка письма и обновление аналитики могут выполняться независимо. Продюсер фиксирует событие, а получатели читают его, когда готовы.

Такой обмен полезен при нескольких независимых получателях, потребности перечитать историю и кратковременных всплесках нагрузки. Синхронный RPC в этих ситуациях заставлял бы отправителя ждать всех участников или самостоятельно хранить незавершенные вызовы.

Взамен появляется незавершенное состояние. Если заказ принят, но резерв еще не создан, API и интерфейс должны это показывать. Событие «заказ оплачен» нельзя публиковать, если пока лишь поставлена команда на проведение оплаты.

Для немедленного решения RPC остается естественным. Проверка доступности операции перед ответом пользователю может потребовать ответа владельца

данных. Перенос ее на Kafka с последующим ожиданием ответного топика способен усложнить ту же синхронную зависимость.

Выбор зависит не только от задержки. Нужно учитывать порядок, повтор, срок хранения сообщений, состояние консьюмера и возможность восстановить процесс. Если прием события связан с изменением базы, эту связь обеспечивают Outbox, записью будущего события в одной транзакции с данными, или другим надежным механизмом. Outbox подробно разобран в разделе паттернов.

4.4 Как проектировать контракты: JSON vs Protobuf, бинарные протоколы? (API First, CodeFirst подходы)

Контракт сначала определяет смысл операции. Какие данные нужны, что считается успехом, какие есть ошибки и можно ли безопасно повторить запрос. JSON и Protobuf задают форму передачи, но не заменяют эти договоренности.

JSON читается человеком, хорошо поддерживается браузерами и легко исследуется обычными HTTP-инструментами. Однако ему нужны явно согласованные типы, правила дат, денежных сумм, отсутствующих и null-полей. OpenAPI и JSON Schema помогают зафиксировать форму.

Protobuf задает типизированные сообщения и номера полей. В бинарной форме обычно передаются числовые теги, поэтому переименование поля не равно переиспользованию его номера. Для ProtoJSON действуют дополнительные ограничения имен и представления значений.

PROTO3

```
syntax = "proto3";
package orders.v1;

message GetOrderRequest {
  string order_id = 1;
}
message GetOrderResponse {
  string order_id = 1;
  string status = 2;
}
service Orders {
  rpc GetOrder(GetOrderRequest) returns (GetOrderResponse);
}
```

Это самостоятельный учебный proto3-контракт. Для Java-кода нужны protoc и плагин генерации Java-кода для gRPC совместимых версий. Генерация не создает бизнес-реализацию сервера.

API First, проектирование от контракта, означает, что контракт обсуждают и версионируют до реализации. Клиент и сервер могут готовиться параллельно. Code First, получение контракта из кода, означает, что описание получают из кода реализации или его интерфейсов. Такой подход быстрее при локальных изменениях, но легко случайно опубликовать внутреннюю модель.

В обоих случаях контракт проверяют при автоматической проверке изменений перед выпуском и сравнивают с предыдущими версиями. Для независимых команд API First часто удобнее как способ согласовать ожидания заранее, а не как формальная обязанность хранить определенный файл.

4.5 Какие плюсы/минусы стриминга в gRPC (server/client/bidi-streaming)?

Обычный одиночный вызов, unary RPC, принимает одно сообщение и возвращает одно. Потоковый вызов меняет количество сообщений в рамках вызова.

Вид	Поток от клиента	Поток от сервера	Пример
Server streaming	Одно сообщение	Несколько сообщений	Постепенная выдача результатов поиска
Client streaming	Несколько сообщений	Одно сообщение	Загрузка набора измерений с итогом
Bidirectional	Несколько сообщений	Несколько сообщений	Интерактивный обмен обновлениями

При двусторонней потоковой передаче стороны могут читать и писать независимо. Порядок сообщений сохраняется в каждом направлении одного RPC, но это не общий порядок всех вызовов системы.

Потоковая передача уменьшает потребность держать большой результат целиком и позволяет отдавать данные по мере готовности. Управление потоком, flow control, позволяет получателю ограничивать скорость поступления данных, однако приложение все равно должно ограничивать свои очереди и не накапливать бесконечно сообщения до отправки.

Длинное соединение удерживает ресурсы и усложняет балансировку. Оборванный поток сообщений не возобновляется автоматически с правильного бизнес-места. Если нельзя терять события, нужны номера сообщений, сохраненная позиция чтения или подтверждения на уровне приложения и процедура восстановления.

Для обычного короткого чтения одиночный вызов проще. Потоковый вызов оправдан, когда данные действительно поступают постепенно или нужен продолжительный обмен. Сам факт большой JSON-строки еще не требует двусторонней потоковой передачи.

4.6 Как обеспечивать обратную совместимость при изменении контрактов?

Совместимость нужно рассматривать в двух направлениях. Новый сервер должен обслуживать старого клиента, а новый обработчик должен понимать старые данные. Для событий, которые хранятся долго, второе особенно существенно.

В HTTP нельзя незаметно делать необязательный параметр обязательным, менять единицы суммы, сужать допустимые значения или заменять привычный смысл кода ответа. Дополнительное поле в JSON допустимо, если клиенты согласованно игнорируют неизвестное.

В Protobuf сохраняют номера и совместимые типы полей. Удаленный номер резервируют. Новое значение перечисления enum тоже требует продуманной обработки неизвестных значений. Совместимость двоичного формата не гарантирует прежнюю бизнес-логику.

Развертывание выполняют поэтапно. Сначала получатели учатся понимать новое представление, затем отправители начинают его использовать. Старую форму убирают после подтвержденной миграции. Если новые поля обязательны для нового поведения, для старых сообщений задают понятный путь обработки.

Контрактные тесты и автоматическое сравнение схем ловят структурные изменения. Для смысла полезны сохраненные примеры запросов и событий, включая исторические версии и неизвестные значения. Даже совместимый контракт может создать несовместимую нагрузку, если новый клиент начинает делать в десять раз больше запросов.

4.7 Как выбирать паттерн коммуникации: запрос-ответ, fire-and-forget, pub/sub?

Запрос-ответ нужен, когда отправитель должен получить конкретный результат. Это может быть чтение заказа или попытка создать резерв. Ответ должен отличать завершенный результат от приема задачи к обработке.

Отправка без ожидания результата, fire-and-forget, означает, что отправитель не ждет бизнес-ответ. Но полезно отдельно определить, ждет ли он надежного подтверждения приема. Запись команды в долговременную очередь с подтверждением и отправка в память другого процесса дают очень разные риски.

Публикация и подписка, pub/sub, подходят для сообщения о произошедшем факте. Сервис заказов публикует `OrderCreated`, а склад, аналитика и уведомления реагируют независимо. Продюсер не должен обязательно знать полный список подписчиков.

Команда и событие отличаются намерением. `ReserveStock` просит владельца склада выполнить действие. `StockReserved` утверждает, что действие уже произошло. Если команда попала в брокер, она еще не становится событием успешного результата.

Паттерн не жестко привязан к одному протоколу. Запрос-ответ можно построить поверх брокера с идентификатором запроса, который связывает его с ответом, и отдельным каналом ответов, а публикацию и подписку поддерживают разные системы. Но для ожидания немедленного ответа такой обмен сложнее, чем прямой RPC. Выбирают понятную модель ответственности и подтверждения.

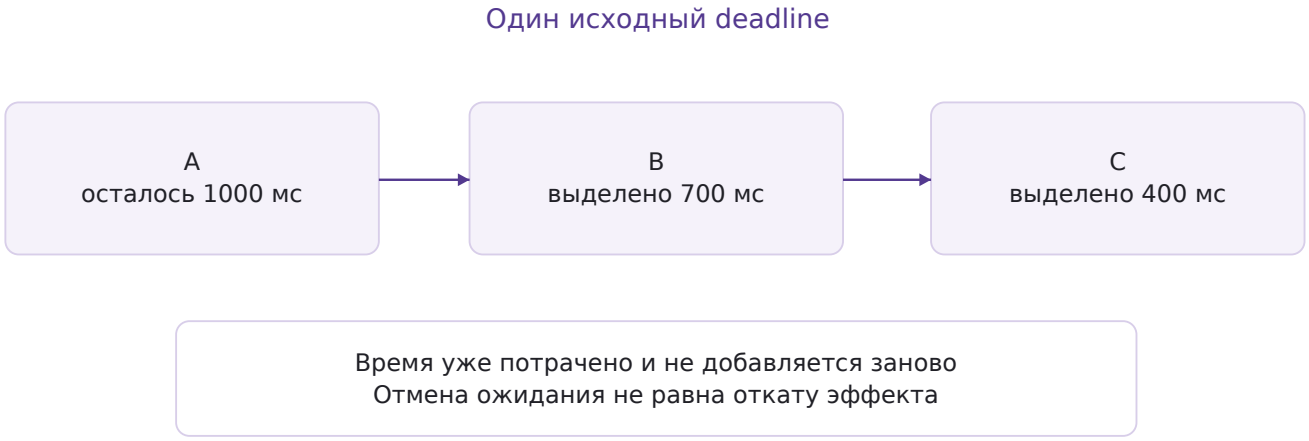
4.8 Как решать проблему каскадных таймаутов в синхронных цепочках?

Пользователь готов ждать одну секунду. Сервис A вызывает B, а B вызывает C. Если каждый получает независимый таймаут в одну секунду и несколько ретраев, общий путь легко выходит за обещанный срок.

В цепочке передают общий крайний срок завершения, deadline, или оставшееся время. Каждый следующий вызов получает не больше оставшегося времени с запасом на обратный путь и собственную обработку. Для gRPC этот срок является частью модели вызова. В HTTP бюджет обычно передают договоренным способом и применяют в клиенте.

Например, A оставляет B 700 мс, B расходует 200 мс и дает C не более 400 мс. Это учебные числа. Главное состоит в уменьшении бюджета, а не в выборе универсальных значений.

Отдельно ограничивают установление соединения, ожидание свободного соединения и чтение ответа. Один таймаут чтения не гарантирует общий предел всей операции, особенно при повторах и поиске IP-адреса сервиса по имени через DNS.



Если клиент перестал ждать, удаленная операция могла продолжить выполнение и завершиться успешно. Отмена должна распространяться по цепочке, где это поддерживается, а приложение должно ее учитывать. Однако отмена транспорта не является откатом платежа. Для изменяющей операции сохраняют ключ идемпотентности и возможность проверить итоговый статус.

Ретраи выбирают на одном согласованном уровне. По три попытки на каждом из трех уровней могут дать 27 обращений к самому нижнему сервису для одной исходной операции.

4.9 Что важно учесть для идемпотентности при повторной доставке?

Повтор должен сохранять идентичность исходной операции. Если после каждого таймаута клиент генерирует новый ключ, сервер видит новые команды и не может распознать, что это один платеж.

Нужно определить область ключа. Обычно это сочетание организации клиента, вида операции и идентификатора команды. Содержание значимых полей связывают с ключом через сохраненные данные или отпечаток запроса, fingerprint. Он рассчитывается по значимым полям, например сумме и получателю, чтобы повтор с другим телом не получил чужой результат.

Проверка «такой ключ уже есть?» и выполнение эффекта не должны быть двумя независимыми действиями без защиты от конкуренции. Два экземпляра могут одновременно увидеть отсутствие строки. Уникальное ограничение и локальная транзакция дают надежную точку сериализации.

Для сообщений различают eventId, идентификатор события, и идентификатор самой бизнес-операции. Первое отличает транспортное событие, второе защищает смысл операции, если ее случайно опубликовали повторно как новое событие. Обработчик может также проверять ожидаемую версию состояния.

Нужно учитывать параллельный повтор, сбой в середине, срок хранения ключа и внешние эффекты. Redis с TTL удобен для некоторых задач, но отдельная Redis-запись и PostgreSQL-транзакция сами по себе не образуют атомарное целое. Гарантию выбирают по цене повторного эффекта и возможному периоду повторного чтения истории.

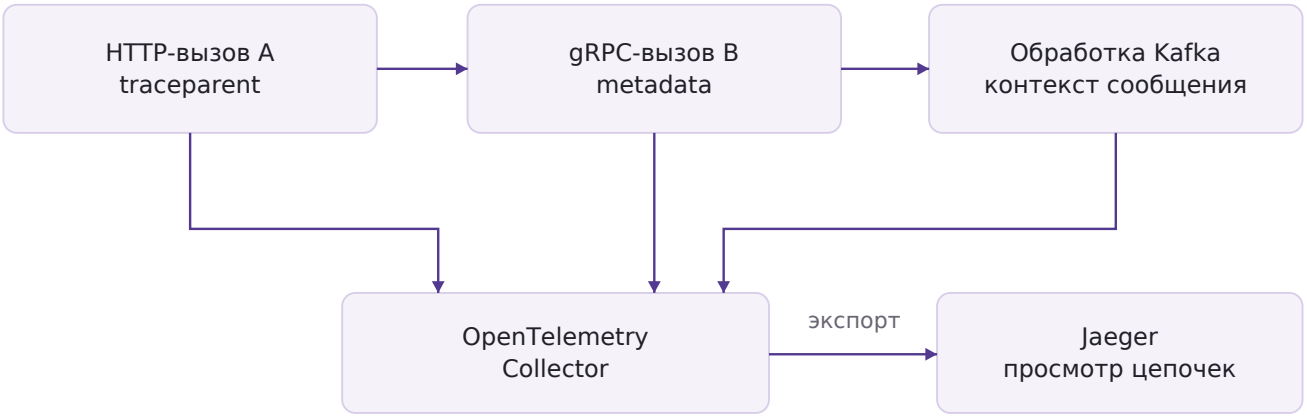
4.10 Как организовать трассировку сквозь REST/gRPC/Kafka (trace-идентификаторы)? (OpenTelemetry + Jaeger)

Трассировка показывает путь одной операции через несколько сервисов. Вся цепочка называется trace. Отдельный участок, например HTTP-вызов или запрос к PostgreSQL, называется span. У цепочки есть общий traceId, а у каждого участка свой spanId и ссылка на предыдущий участок. По этим данным можно восстановить, кто кого вызвал и где потратилось время.

OpenTelemetry предоставляет средства создания, передачи и отправки данных трассировки. При сетевом вызове нужно передать контекст, то есть traceId и другие сведения о текущем участке работы. В HTTP для этого обычно используют заголовок traceparent по стандарту W3C Trace Context. В gRPC сведения идут в метаданных вызова, в Kafka в заголовках сообщения.

При отправке в Kafka библиотека трассировки помещает контекст в заголовки сообщения. Консьюмер извлекает его и создает участок трассировки для обработки. Если один пакет содержит несколько независимых событий, используют связи между участками, span links. Они позволяют показать причинную связь, не выдавая разные операции за одну простую цепочку вызовов.

Контекст связывает работу, экспорт сохраняет spans



Передача контекста и доставка telemetry являются разными потоками

OpenTelemetry Collector принимает метрики, логи и данные трассировки, обрабатывает их и отправляет в выбранные хранилища. Например, Jaeger хранит и показывает цепочки вызовов. Один traceId в строке лога еще не создает трассировку. Сервисы должны действительно записывать участки работы и отправлять их в хранилище.

В Java контекст часто связан только с текущим потоком. Если работа передается в другой поток или выполняется асинхронно, библиотека трассировки должна перенести контекст туда. Это относится и к MDC, набору полей, которые логгер добавляет к записи, например traceId. Иначе фоновые логи перестанут связываться с исходным запросом.

Сохранять можно не все трассировки, а только выбранную часть. Такой отбор называют sampling. Поэтому отсутствие операции в Jaeger иногда означает, что ее просто не отобрали. Дополнительные поля контекста, baggage, не должны содержать секреты и персональные данные. Внешний traceId помогает связать наблюдения, но не подтверждает права пользователя.

4.11 Как бороться с «шумом» и потерями при массовых ретраях?

При массовом отказе одновременные повторы увеличивают нагрузку именно тогда, когда система уже не справляется. Короткий сбой способен превратиться в длительный шквал повторных запросов.

Ретраи ограничивают числом попыток и общим временем. Между ними применяют увеличивающиеся задержки между попытками со случайным разбросом, jitter. Тогда клиенты не возвращаются ровно в одну миллисекунду. Временные ошибки отличают от окончательных. Неверный запрос обычно не исправится от повтора.

Полезно ограничивать долю повторных попыток относительно исходных запросов. Это называют бюджетом повторов. Ограничитель частоты защищает от слишком большого потока, bulkhead ограничивает число одновременных операций, а Circuit Breaker временно прекращает вызовы неисправной зависимости. Эти механизмы

подробно разобраны в следующем разделе. Для HTTP 429 и 503 учитывают указанный сервером срок до следующей попытки, Retry-After.

В брокере не нужно мгновенно возвращать одну ошибочную запись в бесконечный цикл. Используют ограниченные попытки, управляемые топики отложенных повторов или паузу, а затем разбор DLQ. При этом учитывают изменение порядка событий.

Шум в логах уменьшают агрегацией одинаковых ошибок, сохраняя счетчики и итоговый контекст. Но retry не должен тихо завершаться успехом после исчерпания попыток. Для принятой бизнес-команды остается явное состояние ошибки или ожидания и путь восстановления. Это помогает избежать настоящих потерь, скрытых за тишиной в логах.

4.12 Когда уместен API Gateway, а когда BFF, и как это влияет на протоколы?

API Gateway удобен как общая входная точка. Он маршрутизирует, проверяет данные доступа, применяет квоты, TLS и общие сетевые политики. Несколько клиентских приложений могут использовать одну такую точку.

BFF, backend for frontend, проектируется под конкретный интерфейс. Мобильному приложению нужен компактный ответ с минимумом сетевых переходов, а веб-интерфейсу может понадобиться другая композиция. BFF собирает нужную форму данных и отвечает за требования этого клиента.

Оба подхода могут сосуществовать. Общий gateway проверяет запрос и направляет его в BFF мобильного приложения, а BFF обращается к доменным сервисам. Иногда для небольшого продукта достаточно одного слоя, если его ответственность остается понятной.

Внешний HTTP/JSON и внутренний gRPC могут сочетаться. При преобразовании нужно согласовать ошибки, крайний срок выполнения, авторизацию и ограничения размера. Нельзя механически отобразить все gRPC-статусы одним HTTP 500.

Kafka обычно используется за границей синхронного ответа. BFF может принять создание операции и вернуть ее id, но не должен обещать завершение только потому, что команда отправлена в топик. Доменные правила остаются у владельцев соответствующих данных.

5. Паттерны микросервисов

5.1 Что такое API Gateway и зачем он нужен? (и остальные паттерны микросервисов. ВСЕ!)

API Gateway принимает внешние запросы и направляет их к нужным внутренним сервисам. Клиенту не нужно знать адрес каждого внутреннего приложения. В одной точке удобно применять проверку токена, ограничения частоты, защиту соединений через TLS и общую маршрутизацию.

Gateway сам становится важной зависимостью. Его запускают с резервированием, ограничивают ресурсы и наблюдают за задержкой. Если он начинает выполнять все бизнес-правила и хранить состояние всех доменов, получается новый центр связанности.

Единого закрытого списка «всех паттернов микросервисов» не существует. Паттерн описывает повторяющуюся задачу и способ ее решения. Ниже приведена карта основных семейств, а сложные механизмы разобраны в следующих вопросах.

Семейство	Основные паттерны	Какую проблему решают
Границы	Разделение по бизнес-возможностям и ограниченным контекстам	Делят ответственность по бизнес-смыслу
Данные	Свои данные у каждого сервиса, сборка ответа из нескольких API	Ограничивают прямое вмешательство в чужие данные
Вход	API Gateway, BFF, сборка общего ответа	Дают клиенту управляемую точку входа и форму ответа
Поиск сервисов	Поиск адресов сервисов на стороне клиента или сервера	Находят живые экземпляры при изменении адресов
Доставка событий	Outbox и Inbox, связь сообщения с транзакцией	Закрывают окна между изменением данных и сообщением
Процессы	Saga, TCC, координатор бизнес-процесса	Управляют операцией через несколько владельцев данных
Работа с состоянием	CQRS, Event Sourcing, подготовленные представления для чтения	Разделяют модели и способы восстановления данных
Устойчивость	Таймауты, повторы, Circuit Breaker, ограничение параллельности	Ограничивают последствия задержек и отказов

Семейство	Основные паттерны	Какую проблему решают
Нагрузка	Ограничение частоты, отказ от лишней работы, замедление отправителя	Не дают принять больше работы, чем можно выполнить
Миграция	Постепенная замена системы, слой совместимости со старой моделью	Постепенно заменяют старую систему и переводят ее модель
Развертывание	Постепенное обновление, пробный выпуск на части трафика, переключение между двумя версиями	Управляют риском выпуска новой версии
Инфраструктурное расширение	Дополнительный контейнер рядом с приложением, сетевой посредник, service mesh	Выносят повторяющиеся функции рядом с приложением или в сеть
Наблюдаемость	Трассировка вызовов, общий поиск логов, проверки состояния	Помогают связать симптомы и восстановить работу

Обнаружение сервисов, service discovery, позволяет найти работающие экземпляры по имени, не зная их меняющиеся IP-адреса. В Kubernetes эту задачу могут выполнять Service и DNS, поэтому отдельный реестр адресов нужен не всегда. Защитный слой между старой и новой моделью, anti-corruption layer, переводит чужие структуры и коды в понятные новой системе значения.

Backpressure означает, что получатель сообщает отправителю о нехватке свободного места или скорости обработки, чтобы тот замедлил отправку. Load shedding означает осознанное отклонение части работы при перегрузке. Агрегация собирает один ответ из нескольких сервисов. Для нее заранее решают, сколько ждать и что вернуть, если один сервис не ответил.

Это набор инструментов. Применение каждого требует собственной причины. Например, Event Sourcing не является обязательным дополнением к Saga, а CQRS не требует отдельного брокера.

5.2 Чем BFF отличается от «просто» API Gateway и когда его вводить?

Общий gateway отвечает прежде всего за вход и сетевые политики. BFF отвечает за удобство конкретного клиентского приложения. Различие находится в ответственности, а не обязательно в числе процессов.

Представим экран мобильного банка. Ему нужны последние операции, краткий баланс и признак доступности платежа. Если мобильный клиент самостоятельно делает шесть вызовов по нестабильной сети, BFF может собрать компактный ответ на сервере.

Для веб-интерфейса набор данных и сценарии могут быть другими. Отдельный BFF позволяет команде менять их без превращения общего API в множество флагов «если это мобильное приложение». При этом дублирование бизнес-правил между BFF создает риск расхождений.

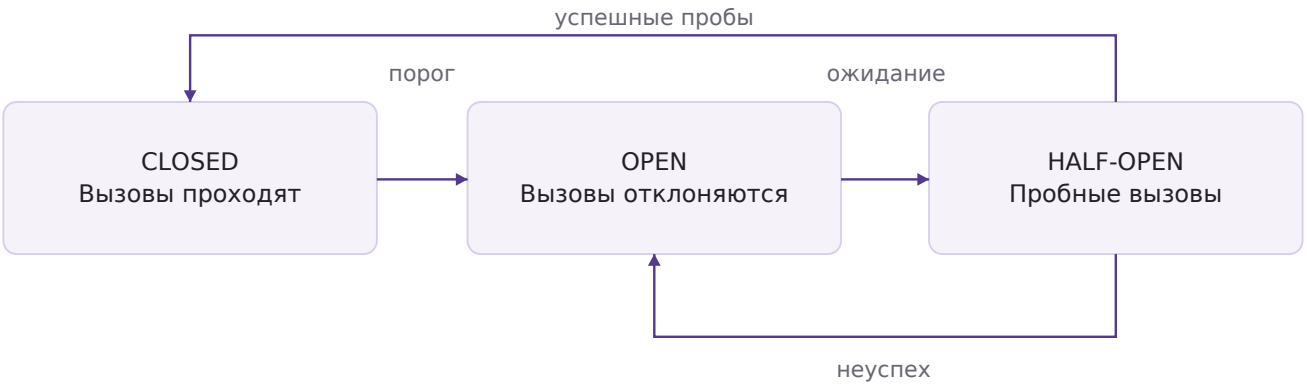
В BFF уместны композиция, адаптация формата и логика представления. Решение о допустимости платежа остается у платежного сервиса. Кэширование и частичные ответы допустимы, если интерфейс умеет правильно их показывать.

Вводить BFF полезно, когда различия клиентов уже создают измеримую сложность. Для одного простого интерфейса отдельный слой может лишь добавить задержку и эксплуатационные расходы. Gateway и BFF не обязаны появляться вместе в первом релизе.

5.3 Что такое Circuit Breaker и когда он срабатывает?

Circuit Breaker отслеживает результаты вызовов конкретной зависимости и временно прекращает обращения, когда они слишком часто завершаются ошибкой или выполняются слишком долго. Это дает системе возможность не тратить все ресурсы на заведомо проблемный путь.

В закрытом состоянии, closed, вызовы проходят. Circuit Breaker считает результаты за последние несколько вызовов или за период времени. При превышении допустимой доли ошибок он переходит в открытое состояние, open, и быстро отклоняет новые вызовы. Позже состояние half-open разрешает несколько пробных обращений. Если зависимость восстановилась, обычные вызовы возобновляются.



Статистика срабатывает после minimumNumberOfCalls

В Resilience4j учитывают размер и тип окна, minimumNumberOfCalls, минимальное число наблюдений, порог ошибок, порог медленных вызовов, длительность open и число проб в half-open. Не каждая бизнес-ошибка должна считаться отказом зависимости. Ответ «недостаточно средств» может означать, что платежный сервис полностью исправен.

Circuit Breaker сам по себе не прерывает зависший вызов. Для этого нужен timeout в клиенте или соответствующий ограничитель времени. Breaker также не ограничивает число одновременно выполняемых запросов, это задача bulkhead.

Обычный breaker хранит состояние локально в процессе. У десяти экземпляров может быть десять независимых состояний. Поэтому он не является общим глобальным выключателем и не заменяет защиту самой зависимости.

5.4 Для чего нужны Bulkhead, RateLimiter и Timeouts вместе?

Эти механизмы ограничивают разные стороны нагрузки. Таймаут ограничивает время ожидания. Rate limiter ограничивает частоту новых запросов. Bulkhead ограничивает число одновременно занятых потоков, соединений или других ресурсов. Их совместное применение не дает медленной зависимости занять все приложение.

Допустим, сервис оплаты замедлился. Без timeout каждый вызов долго удерживает поток и соединение. Один timeout помогает освободить ожидающего, но при большом потоке новых запросов ресурсов все равно может не хватить.

Ограничитель частоты уменьшает число новых запросов за период. Но даже редкие очень долгие вызовы могут занять много ресурсов. Bulkhead, ограничитель параллельности, задает максимальное число операций с одной зависимостью. Так неисправная оплата не займет все потоки сервиса. Для реализации можно ограничить число разрешений на вызов, выделить отдельный набор рабочих потоков или отдельный пул соединений.

Очередь перед bulkhead тоже ограничивают. Бесконечная очередь просто переносит отказ из соединений в память и время ожидания. При переполнении полезнее быстро и явно отказать, если бизнес-контракт это допускает.

Порядок применения ограничений влияет на результат. Если запрос долго ждет свободного места в очереди, на сетевой вызов времени может уже не остаться. При повторе можно либо заново занимать место в ограничителе параллельности, либо удерживать его между попытками. Поэтому настройки нужно проверять вместе по времени ожидания, числу занятых потоков и поведению при отказе.

5.5 Что такое Retry-стратегии и как избежать thundering herd?

Retry выполняет еще одну попытку после неуспеха. Он полезен для кратких сетевых сбоев и смены лидера, но способен повторить уже состоявшуюся бизнес-операцию. Поэтому сначала выясняют, безопасен ли повтор и как уточнить неопределенный результат.

Фиксированная задержка проста, но синхронизирует клиентов. Экспоненциальная задержка увеличивает паузу после ошибок. Верхняя граница не дает паузе расти бесконечно. Случайный разброс задержки, jitter, добавляет случайность, чтобы клиенты не возвращались одной толпой. Такая толпа одновременных попыток называется thundering herd.

В одном варианте такого разброса выбирают случайную задержку между нулем и текущим ограниченным экспоненциальным пределом. Точные числа задают по

задаче, а общий крайний срок завершения операции остается важнее возможности сделать еще одну попытку.

Нужно определить повторяемые ошибки. Некорректный запрос и отказ авторизации обычно требуют исправления причины. Таймаут может быть временным, но не доказывает, что сервер ничего не выполнил. Для изменяющих операций требуется ключ идемпотентности.

Ретраи согласуют между клиентом, gateway и сетевым слоем между сервисами. Если каждый слой делает по три попытки, нагрузка на нижний сервис умножается. Метрики должны различать исходные запросы, попытки и окончательные результаты, иначе сервис выглядит загруженным, а бизнес-прогресс почти отсутствует.

5.6 Как работает паттерн Outbox и чем он помогает против «двух записей»?

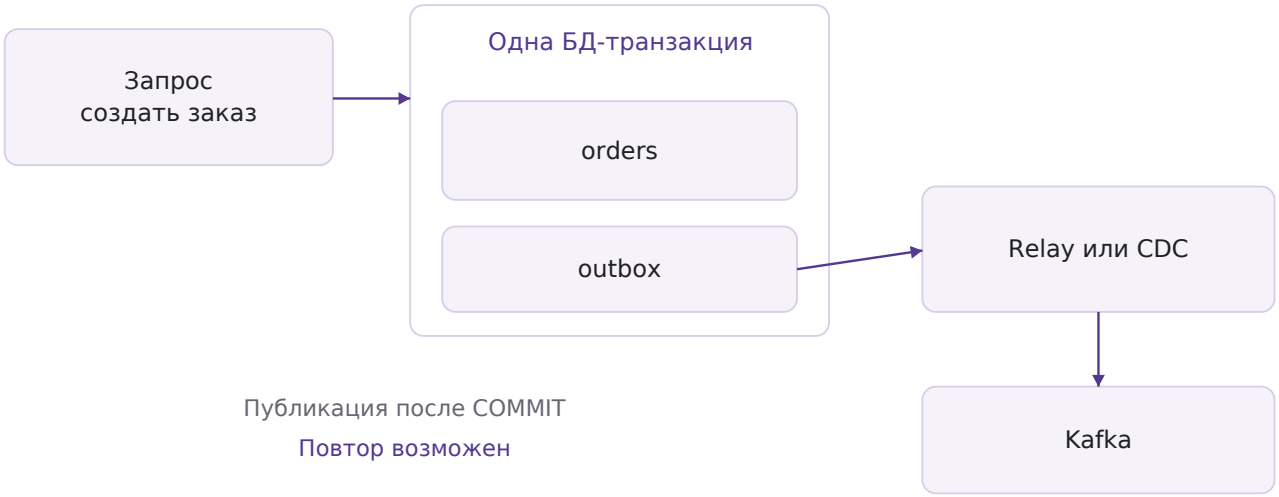
Нужно создать заказ в PostgreSQL и отправить OrderCreated в Kafka. Если выполнить два независимых действия, между ними можно упасть. Сначала база, потом Kafka дает сохраненный заказ без события. Сначала Kafka, потом база дает событие о заказе, который не сохранился.

Outbox сохраняет намерение отправить событие в той же локальной транзакции, что и бизнес-данные. В таблицу orders записывается заказ, в outbox событие с eventId. COMMIT либо сохраняет обе записи, либо не сохраняет ни одну.

POSTGRESQL

```
BEGIN;
INSERT INTO orders (id, status)
VALUES (:orderId, 'CREATED');
INSERT INTO outbox (event_id, aggregate_id, event_type, payload)
VALUES (:eventId, :orderId, 'OrderCreated', :payload);
COMMIT;
```

Это фрагмент для PostgreSQL с параметрами приложения. Таблицы должны существовать, event_id быть уникальным, а payload соответствовать выбранному типу, например jsonb.



Отдельный фоновый отправитель читает outbox и публикует события в Kafka. Его часто называют relay. Он может опрашивать таблицу или получать уже зафиксированные изменения из журнала базы. Второй способ называется CDC, отслеживание изменений данных. Например, Debezium умеет читать журнал PostgreSQL, а его Outbox Event Router преобразует записи outbox в сообщения.

После подтверждения брокером отправитель отмечает доставку или сохраняет позицию, с которой нужно продолжить после сбоя. Если приложение упадет между отправкой и отметкой, событие придет повторно. Поэтому **Outbox предотвращает потерю события между сохранением бизнес-данных и публикацией, но не устраняет все дубликаты**.

Если отправителей несколько, нужно не дать им одновременно забрать одну запись. Для этого используют блокировки или временное закрепление задания за одним исполнителем. Также сохраняют порядок событий одной сущности, если он важен. Следят за размером outbox и возрастом самой старой неотправленной записи. Удалять записи можно после подтвержденной доставки с учетом способа восстановления.

5.7 Что такое Transactional Inbox/Outbox и в чём их различие?

Outbox отвечает за исходящее событие. Он связывает изменение собственных данных с надежным намерением сообщить о нем. Inbox отвечает за входящую команду или событие и защищает ее прием и обработку.

Простой transactional Inbox похож на таблицу processed_event из раздела о консьюмере. Уникальный идентификатор входящего сообщения и бизнес-эффект записываются в одной транзакции. При повторе факт выполнения не позволяет повторно применить действие.

Другой вариант Inbox сначала надежно сохраняет принятое сообщение, затем фоновый обработчик обрабатывает его по отдельному протоколу. В этом случае нужно различать «принято» и «выполнено», хранить статус и корректно

восстанавливать незавершенную работу. Одна запись в Inbox еще не доказывает бизнес-успех.

Оба паттерна можно использовать вместе. Транзакция проверяет входящий eventId, меняет состояние заказа и добавляет исходящее событие в Outbox. После COMMIT входная позиция чтения подтверждается, а фоновый отправитель позднее отправляет результат.

Это дает понятную атомарную границу в одной базе. Внешнее списание денег не попадет в нее просто от добавления двух таблиц. Для внешнего эффекта по-прежнему нужен поддерживаемый им ключ операции или отдельный процесс согласования.

5.8 Когда уместен Event Sourcing и какие у него подводные камни?

При обычном хранении мы сохраняем текущее состояние заказа. При Event Sourcing источником истины становится последовательность событий, из которой это состояние получается. Например, OrderCreated, ItemAdded, PaymentAccepted и OrderCancelled.

События здесь являются записью фактов предметной области. Для чтения текущего состояния обработчик последовательно применяет их к модели. Снимок текущего состояния, snapshot, может ускорить восстановление, но сам поток событий остается основой истины.

При одновременных изменениях проверяют ожидаемую версию агрегата. Агрегат объединяет данные и бизнес-правила, которые должны меняться согласованно, например заказ вместе с его позициями. Если обработчик прочитал версию 7, а другой уже сохранил 8, первый не должен молча записать изменение поверх нее. Он должен обнаружить конфликт и решить его.

Event Sourcing полезен, когда важны объяснение истории, восстановление прошлых состояний и сложная событийная модель. Обычная таблица аудита тоже может покрыть часть этих задач дешевле.

Цена существенна. Старые события придется читать новым кодом, исправлять ошибочные факты новыми событиями и поддерживать проекции. При повторном проигрывании истории нельзя повторно отправлять реальные письма и платежи. Изменившиеся внешние справочники и текущее время способны сделать старый расчет невоспроизводимым, если нужный контекст не сохранен.

Удаление персональных данных и длительное хранение тоже нужно проектировать заранее. Наличие событий в Kafka не означает Event Sourcing. Если истина находится в изменяемых строках PostgreSQL, а Kafka получает уведомления, модель хранения остается другой.

5.9 Что такое CQRS и зачем разделять чтение и запись?

CQRS разделяет модель выполнения команд и модель чтения. Команда изменяет состояние с проверкой бизнес-правил, запрос возвращает нужное представление без изменения состояния.

Например, оформление заказа проверяет допустимые переходы и резерв. А экран истории хочет плоский список с названием товара, итоговой суммой и статусом доставки. Одной моделью неудобно одновременно выражать правила записи и быстро собирать этот экран.

Простейший CQRS может существовать в одном приложении и одной базе. Для записи используются методы с бизнес-правилами, для чтения отдельные SQL-запросы и DTO. Отдельные сервисы, Kafka и разные базы не обязательны.

При более сильном разделении события обновляют подготовленную модель чтения, materialized view. Это сохраненное представление, например таблица с уже собранными данными для экрана. Такая таблица может быть оптимизирована под конкретный экран или поиск. Тогда появляется задержка между успешной командой и видимостью в проекции.

API и интерфейс должны учитывать эту задержку. Например, после команды возвращают id и версию результата, а экран временно показывает принятое состояние или читает владельца записи. Проекцию нужно уметь восстановить и проверить на пропуски.

CQRS и Event Sourcing независимы. Первое разделяет модели чтения и записи, второе определяет способ хранения истины. Если обычные операции создания, чтения, изменения и удаления данных уже удобны, дополнительная проекция может принести больше синхронизации, чем пользы.

5.10 Двухфазная фиксация (2PC): как работает и почему редко подходит в микросервисах?

Двухфазная фиксация позволяет нескольким транзакционным участникам согласовать одно решение о COMMIT или ROLLBACK. Для этого нужен координатор и участники с поддержкой протокола, например через XA, интерфейс участия ресурсов в общей транзакции.

В первой фазе координатор просит подготовиться. Каждый участник выполняет свою работу и надежно записывает подготовленное состояние, prepared, из которого он способен позже завершить транзакцию. После ответа «готов» участник уже не должен самостоятельно забыть обещание.

Во второй фазе координатор принимает общее решение и надежно сохраняет его, затем сообщает участникам. Если хотя бы один не подготовился, выполняется rollback. Если все подготовились и решение commit принято, участники должны завершить commit, включая восстановление после сбоев.



Подготовленные участники ждут общего решения

Сложность проявляется между фазами. Подготовленный участник может удерживать блокировки и ресурсы, пока не узнает решение. Недоступный координатор или сеть способны надолго оставить транзакцию в неопределенном для участника состоянии. Надежное восстановление является частью протокола.

Обычный HTTP-эндпоинт не становится участником 2PC от того, что его дважды вызвали. PostgreSQL имеет PREPARE TRANSACTION для внешнего координатора, но обычному приложению без него оставлять подготовленные транзакции опасно.

Для микросервисов с разными базами и долгими шагами общая координация часто слишком сильно связывает доступность и эксплуатацию. 2PC при этом не является запрещенным или всегда неверным решением. В ограниченной среде совместимых участников он может быть уместен. Saga и TCC меняют модель процесса и гарантий, а не являются бесплатной реализацией той же глобальной транзакции.

5.11 Saga: оркестрация vs хореография — плюсы/минусы, где какую взять?

Saga разбивает бизнес-процесс на локальные транзакции. После каждого шага его изменение уже зафиксировано. Если дальнейший шаг не удался, выполняются отдельные компенсирующие действия.

Для заказа можно создать резерв товара, принять платеж и оформить доставку. Если доставка недоступна окончательно, процесс может освободить резерв и запустить возврат. Между шагами система находится в промежуточном состоянии. **Saga не дает общей изоляции как одна локальная транзакция базы с атомарностью и изоляцией.**

При оркестрации отдельный координатор хранит состояние процесса и отправляет команды участникам. Видно, какой шаг выполняется, какие попытки уже были и когда нужна компенсация. Цена состоит в дополнительном компоненте и риске сосредоточить в нем слишком много доменной логики.

При хореографии сервисы реагируют на события друг друга. После StockReserved оплата начинает свою работу и публикует результат. Для короткой независимой

цепочки это может быть естественно. С ростом числа участников причинные связи и восстановление становятся труднее отслеживать.

Пример бизнес-процесса с отказом доставки



Компенсации могут повторяться и не стирают историю

Вариант	Удобен, когда	Основной риск
Оркестрация	Много шагов, таймеров, ветвлений и компенсаций	Слишком сложный центральный координатор
Хореография	Небольшое число независимых реакций на факты	Скрытая цепочка событий и циклические зависимости

В обоих вариантах нужны надежная доставка, идемпотентность шагов, timeout процесса и хранение контекста. Координатор не должен помнить незавершенную Saga только в оперативной памяти. Сбой во время компенсации тоже требует повторов и иногда ручного решения. Возврат денег может занимать время, поэтому «откат» не означает мгновенно стереть все следы.

5.12 TCC (Try-Confirm/Cancel): чем отличается от Saga и когда проще внедрить?

TCC означает Try, Confirm и Cancel. Каждый участник предоставляет три бизнес-операции. Try проверяет возможность и создает резерв, Confirm окончательно использует его, Cancel освобождает.

Для кошелька Try может зарезервировать сумму, не делая ее доступной для других платежей. Для склада он резервирует количество. Если все резервы созданы, координатор подтверждает их. При неуспехе освобождает уже созданные.

В Saga шаг обычно уже совершает локальное бизнес-изменение, а компенсация выполняет встречное действие. В TCC участник специально проектирует промежуточное состояние резерва. Это может уменьшить потребность возвращать уже необратимо использованный ресурс, но требует явных API и учета резервов во всех конкурирующих операциях.

TCC легче внедрить, когда бизнес уже знает резервы, например блокировка суммы на счете или бронирование товара, а участники находятся под нашим контролем. Если внешний провайдер умеет только безвозвратно выполнить действие, добавить Try поверх него без изменения смысла не получится.

Confirm и Cancel должны выдерживать повторы. Нужно обрабатывать пустой Cancel, когда Try не состоялся, и поздний Try после отмены. Иначе отмененный процесс неожиданно создаст новый резерв. Идентификатор транзакции и финальное состояние защищают от такого «подвешенного» выполнения.

У резервов есть время жизни. Его истечение должно быть согласовано с координатором, который мог уже принять решение Confirm. TCC не равен 2PC базы и не обещает глобальную изоляцию общей транзакции автоматически. Финальные бизнес-команды могут доставляться и завершаться не одновременно.

5.13 Как проектировать компенсирующие операции и хранить их контекст?

Компенсация исправляет бизнес-последствие уже завершенного шага. Она не возвращает процесс к прошлому состоянию памяти и не делает историю несостоявшейся.

Если клиенту уже отправили письмо, «удалить факт отправки» недостаточно. Иногда компенсацией будет уточняющее письмо. Если деньги списаны, возвращают сумму отдельной операцией со своими сроками и идентификатором.

Контекст сохраняют до того, как он понадобится для восстановления. Для резерва нужен reservationId, для возврата paymentId, сумма, валюта и причина. Повторный расчет по текущим данным опасен. Цена и курс могли измениться после первоначального шага.

У каждого шага должны быть устойчивые состояния, идентификатор операции, журнал попыток и понятные разрешенные переходы. Например, созданный резерв можно освободить, уже освобожденный считается успешно отмененным, а подтвержденный может потребовать другой бизнес-процедуры.

Компенсации тоже падают и повторяются. Их идемпотентность и хранение результата так же важны, как для прямых действий. Часто компенсируют в обратном порядке, но бизнес-зависимости могут потребовать другой последовательности.

Если автоматическое исправление невозможно, процесс остается в явном состоянии, например COMPENSATION_FAILED, с причиной и инструкцией для оператора. Нельзя просто пометить Saga отмененной и скрыть незавершенный возврат.

5.14 Как выбирать ключи идемпотентности и TTL для хранения факта выполнения?

Хороший ключ отвечает на вопрос «какую одну команду клиент пытается выполнить». Он создается до первого вызова и сохраняется во всех повторах. Для нового платежа нужен новый ключ, даже если сумма и получатель совпали.

UUID подходит как технический идентификатор команды, если клиент умеет надежно его сохранить. В некоторых доменах лучше существует естественный идентификатор, например номер заявки внешней системы. Один хеш всего JSON плох как единственное правило, потому что два законных одинаковых платежа могут стать неразличимы.

Ключ ограничивают областью организации клиента и видом операции. Отпечаток тела нужен для проверки, что повтор действительно относится к прежнему содержимому. Авторизация проверяется независимо, знание ключа не дает права читать результат.

TTL, срок хранения записи о выполнении, выбирают по максимальному времени возможного повтора, хранения команд и восстановления. Если сообщение можно повторно прочитать из истории через месяц, ключ на сутки защищает только первые сутки. После удаления записи поздний повтор может снова выполнить эффект.

Для критичных денег полезно иметь постоянное бизнес-ограничение, например уникальный внешний paymentId в реестре операций, даже если технический кэш ответов очищается раньше. Сроки хранения статуса, полного ответа и чувствительных данных могут различаться.

Если ключ имеет срок действия, контракт должен объяснять поведение после его истечения. Молчаливое обещание бессрочной защиты при конечном TTL технически неверно.

5.15 Как решать дедупликацию событий при «ровно-один-раз» бизнес-уровнем?

На бизнес-уровне цель обычно состоит в единственном эффекте. Транспорт вправе доставить событие повторно, но один и тот же платеж не должен второй раз менять баланс.

Для этого определяют устойчивый operationId и защищают результат уникальным ограничением. В одной транзакции проверяют идентичность операции, записывают ее результат и меняют нужное состояние. Повтор возвращает прежний результат или ничего не меняет.

EventId полезен для дедупликации сообщения, но его недостаточно, если одна бизнес-операция была ошибочно создана двумя разными событиями. Тогда нужна уникальность бизнес-ключа. И наоборот, одна операция иногда законно порождает несколько разных эффектов, поэтому ключ должен включать назначение эффекта.

Для последовательных состояний используют версию агрегата или ожидаемое состояние. Событие версии 8 не должно перезаписать уже примененную версию 10. Но простая проверка «принимать только больше» годится для снимков, а для приращений способна потерять пропущенные изменения. Модель события нужно понимать.

Если эффект выполняется внешней системой, ей передают тот же ключ либо сверяют неопределенный результат до нового действия. Невозможно закрыть окно между двумя независимыми системами одной локальной строкой processed_event.

Поэтому обещание «ровно один раз» всегда сопровождают границей. Например, «одна проводка на operationId в этой PostgreSQL-базе». Такая формулировка проверяема и полезнее общей гарантии для всей распределенной системы.

5.16 Когда применять «strangler-fig» для постепенного распила монолита?

Strangler fig позволяет постепенно заменить старую систему. Перед ней появляется управляемая точка маршрутизации, которая направляет часть операций в новую реализацию, а остальные продолжает обслуживать старой.

Например, сначала переносится чтение каталога, затем его изменение. Каждый шаг ограничен конкретной областью, а не требует переписать весь магазин перед первым выпуском.

Самый сложный вопрос касается данных. Для каждой операции нужен понятный владелец записи. Временное копирование в обе стороны без протокола приводит к конфликтам. Репликация изменений, перенос уже накопленных исторических данных и сверка должны иметь наблюдаемый прогресс.

Промежуточный слой совместимости переводит старые коды и структуры в новую модель. Это позволяет не переносить внутренние особенности монолита в каждую новую сущность.

Трафик можно включать постепенно и сравнивать результаты безопасного чтения. Но проверочное параллельное выполнение изменяющих операций не должно дважды списывать деньги. Для возврата на старую реализацию учитывают уже записанные новой версией данные.

Паттерн полезен при большой работающей системе, которую нельзя остановить надолго. Завершение означает удаление старого пути и ненужных переходников. Если обе реализации остаются навсегда, стоимость поддержки может только вырасти.

6. Наблюдаемость

6.1 Для чего нам это все дело?

Пользователь говорит, что платеж не прошел. Чтобы разобраться, нужно узнать, сколько таких ошибок, где закончилась обработка и что происходило внутри. Для этого нужны разные виды наблюдений.

Логи фиксируют отдельные события с контекстом. По ним можно найти конкретную ошибку или переход состояния. Метрики показывают измеряемую картину во времени, например скорость запросов, долю ошибок и занятость соединений. Traces связывают участки одной распределенной работы.

В привычном наборе ELK помогает искать логи, Prometheus хранит и запрашивает метрики, Grafana показывает дашборды, то есть экраны с графиками и показателями. Grafana также может управлять алертами. Spring Boot Actuator предоставляет эксплуатационные эндпоинты, а Micrometer связывает Java-приложение с системой метрик.

Наблюдаемость нужна для ответа на конкретные эксплуатационные вопросы. Если мы собираем миллионы строк, но не можем определить, какие заказы зависли, объема данных недостаточно для полезности.

Сначала выбирают важные пользовательские операции, затем измеряют их успешность и задержку. Технические показатели помогают объяснить причину. Например, рост времени платежа связывается с ожиданием свободного соединения из пула базы данных. Пул хранит уже открытые соединения, чтобы не устанавливать их заново для каждого запроса. Система алертов должна замечать такой эффект до массовых обращений пользователей.

6.2 Какие поля логировать обязательно (traceId, spanId, user, важные ключи домена)?

Универсального списка полей для любой строки не существует. Но для большинства логов серверного приложения нужны время в согласованной зоне, уровень, имя сервиса, окружение, версия приложения и понятное имя события.

TraceId и spanId связывают запись с распределенным trace. Для фоновой задачи, у которой нет входящего HTTP-запроса, может быть собственный trace или идентификатор фонового задания. Отсутствующее поле лучше оставить отсутствующим, чем подставить один общий искусственный traceId всем задачам.

Доменные ключи позволяют найти конкретную работу. Это orderId, paymentId, messageId, tenantId и operationId. Их выбирают по месту логирования. При ошибке нужны тип исключения, причина и цепочка вызовов, в которой возникло исключение там, где он помогает, а не на каждой повторной строке одной цепочки.

```json
{
  "timestamp": "2026-11-01T12:00:00Z",
  "level": "ERROR",
  "service": "payment-api",
  "environment": "production",
  "event": "payment_provider_timeout",
  "operationId": "operation-42",
  "durationMs": 801
}
```

Это учебный пример структурированного события. Поля трассировки добавляет подходящая инструментация, если контекст существует. Структурированный JSON удобнее парсить, чем строку со случайным расположением значений.

Поле user не должно означать полный профиль человека. Логируют только нужный идентификатор с соответствующей защитой. Токены, пароли, полные карточные данные и произвольные тела запросов в обычные логи не попадают. TraceId также не используют как метку Prometheus, иначе почти каждая операция создаст новую временную серию.

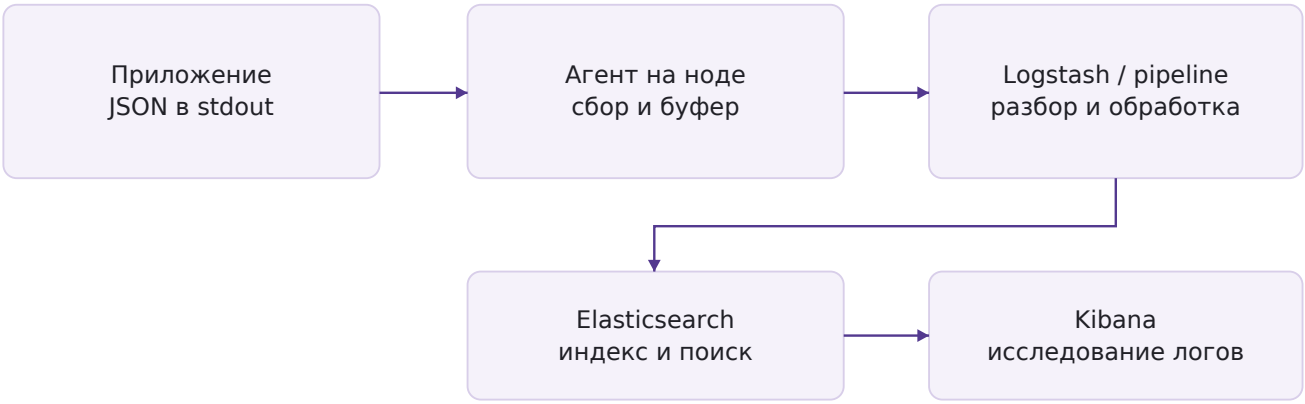
6.3 Как устроена цепочка ELK: кто и за что отвечает (сбор, парсинг, хранение, поиск)?

ELK расшифровывается как Elasticsearch, Logstash и Kibana. Однако реальная цепочка может обходиться без Logstash, если его работу выполняют другие агенты сбора и цепочки предварительной обработки логов.

Приложение пишет структурированные логи в стандартный поток вывода приложения или файл. Агент, например Fluent Bit, Filebeat или Elastic Agent, читает их, добавляет метаданные окружения и передает дальше. В Kubernetes агент часто работает на ноде как DaemonSet.

Logstash принимает события, разбирает, преобразует и маршрутизирует их. При необходимости он использует очередь на диске, которая переживает перезапуск. Часть преобразований можно выполнять цепочка предварительной обработки в Elasticsearch, поэтому несколько одинаковых стадий не обязательны.

Elasticsearch индексирует документы и выполняет поиск и агрегации. Kibana предоставляет интерфейс исследования, dashboards и другие функции работы с сохраненными данными.



Нужно понимать гарантии каждого перехода. Если Elasticsearch недоступен, агент может буферизовать события, пока не закончится заданный ресурс. После переполнения возможна потеря. Дубликаты при повторах тоже зависят от конфигурации и идентификаторов документов.

Обычная цепочка логов не должна быть единственным надежным журналом финансовых операций. Аудит, которому нельзя терять записи, проектируют с отдельными требованиями к сохранности, доступу и восстановлению. Технический лог и бизнес-реестр решают разные задачи.

6.4 Как проектировать индексы и ILM-политику, чтобы не «сжечь» кластер?

В Elasticsearch индекс делится на шарды. Каждый шард хранит часть документов и обслуживает поиск по ним. Шард требует памяти и других ресурсов, даже если данных в нем мало. Поэтому отдельный индекс для каждого пользователя или слишком частое создание новых индексов способны перегрузить кластер множеством маленьких шардов.

Для логов, которые добавляются по времени, удобны потоки данных, data streams. Под одним именем они объединяют несколько индексов. По мере роста Elasticsearch создает следующий индекс, а предыдущий сохраняет для чтения. Такое переключение называют rollover. Его можно запускать по размеру или возрасту, а не создавать новый индекс строго каждый день независимо от нагрузки.

Схема полей, mapping, задает, как Elasticsearch хранит и ищет каждое поле. Например, keyword подходит для точного идентификатора, а text для поиска слов в тексте. Если создавать новое имя поля для каждого заказа, число полей быстро разрастется. Это называют mapping explosion. Номер заказа должен быть значением одного постоянного поля, а не его именем.

ILM автоматически управляет жизнью индексов. Свежие часто читаемые данные находятся на быстром уровне хранения, hot. Старые можно переводить на менее дорогие уровни, warm и cold, а затем удалять. Доступные действия зависят от поставки Elasticsearch. В управляемом Elasticsearch Serverless вместо ILM используется отдельный механизм управления сроком хранения, data stream lifecycle.

Оценку хранения строят по измеренному размеру индексированных данных, входящему потоку, сроку хранения, числу реплик и запасу на создание новых индексов и восстановление. Размер исходного JSON не равен размеру индекса. Реплики увеличивают ресурс и доступность, а резервные снимки решают отдельную задачу восстановления.

Следят за свободным диском, памятью Java, работой сборщика мусора и числом запросов, отклоненных из-за перегрузки. Отдельно смотрят время записи и поиска, состояние шардов и выполнение шагов ILM. Если политика удаления остановилась на ошибке, диск продолжит заполняться, хотя сама политика существует.

6.5 Что такое Spring Actuator и какие эндпоинты включать в проде?

Spring Boot Actuator добавляет эксплуатационные эндпоинты приложения. Они помогают проверить состояние, получить метрики и диагностическую информацию. Это дополнительная поверхность доступа, поэтому ее открывают осознанно.

Эндпоинт health собирает результаты отдельных проверок состояния, например доступность базы и состояние приложения. Для Kubernetes есть отдельные группы liveness и readiness. Эндпоинт metrics показывает зарегистрированные показатели, а prometheus отдает их в формате для Prometheus. Для последнего нужна библиотека, которая умеет преобразовать внутренние метрики приложения в этот формат.

YAML

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus
  endpoint:
    health:
      show-details: never
      probes:
        enabled: true
```

Это минимальный фрагмент application.yml для Spring Boot 4.x. Нужны Actuator и micrometer-registry-prometheus. Открытие эндпоинта через exposure не заменяет сетевую защиту и авторизацию. Prometheus должен иметь разрешенный доступ, а пользователю снаружи эти эндпоинты обычно не требуются.

В актуальном Boot по умолчанию наружу предоставляется только health. Эндпоинт должен быть доступен по настройкам access, разрешенным операциям, и exposure, доступности через HTTP или служебный интерфейс Java JMX, и не блокироваться правилами безопасности приложения. Если приложение определяет собственный SecurityFilterChain, цепочку проверок безопасности, правила Actuator нужно учитывать явно.

Env, configprops, heapdump, threaddump и loggers полезны для диагностики, но могут раскрывать чувствительную информацию или влиять на приложение. Их не открывают всем через `include=*`. Даже скрытие значений не делает все

диагностические метаданные общедоступными. Для отдельного служебного порта отдельно проверяют доступность рабочего порта приложения.

6.6 Как Micrometer/Prometheus собирают метрики из Spring Boot?

Micrometer предоставляет счетчики и другие инструменты измерения в Java-приложении. Counter считает события, Timer измеряет длительность, Gauge показывает текущее значение. Компонент Registry собирает эти показатели и подготавливает их для нужной системы, например Prometheus.

Spring Boot с Actuator автоматически регистрирует многие показатели JVM, HTTP и поддерживаемых компонентов. Для Prometheus добавляют micrometer-registry-prometheus и открывают разрешенный эндпоинт `/actuator/prometheus`.

Prometheus периодически обращается к приложению по HTTP и забирает значения метрик. Один такой опрос называют scrape, а опрашиваемое приложение target. Значения сохраняются во времени. Открыть `/actuator/prometheus` недостаточно, нужно еще сообщить Prometheus адрес приложения и разрешить доступ к нему. ServiceMonitor помогает настроить обнаружение при использовании Prometheus Operator. В обычном Kubernetes этот дополнительный тип ресурса сам по себе отсутствует.

Каждый уникальный набор меток, labels, образует отдельную временную серию. Метки описывают измерение, например HTTP-метод и код ответа. Для `method=POST`, `status=500`, `uri=/orders/{id}` число сочетаний ограничено. Если вместо шаблона пути подставлять каждый настоящий номер заказа, появятся миллионы разных серий. Это называют высокой кардинальностью. Она увеличивает расход памяти и стоимость хранения.

Timer не обязан сразу отдавать распределение длительностей. Для гистограммы нужно настроить интервалы времени, buckets. Например, отдельно считать запросы быстрее 100, 500 и 1000 мс. Порог целевого времени ответа и способ вычисления процентилей тоже задают отдельно. Поэтому запрос к `_bucket` работает только тогда, когда приложение действительно публикует такие серии.

При перезапуске приложения локальные счетчики обнуляются. Функция rate в Prometheus рассчитывает скорость роста, а increase прирост за период. Они умеют учитывать сброс счетчика. Для коротких фоновых задач периодический опрос может быть неудобен. Приложение способно завершиться до того, как Prometheus успеет забрать метрики, поэтому для него выбирают отдельный способ публикации.

6.7 Какие типы метрик Prometheus (counter/gauge/histogram/summary) и для чего каждый?

Тип метрики выбирают по тому, что измеряют. Ошибка в выборе дает красивый график с неверным смыслом.

Тип	Поведение	Пример
Counter	Растет до сброса процесса	Число завершенных платежей
Gauge	Может увеличиваться и уменьшаться	Текущее число занятых соединений
Histogram	Считает распределение наблюдений	Время обработки запросов
Summary	Считает count, sum и необязательные локальные процентили	Длительность с процентилями, вычисляемыми клиентом

Counter читают через скорость или прирост за период. Значение «уже 500 тысяч запросов» мало говорит о текущей нагрузке. Gauge показывает состояние сейчас, но не дает надежную историю числа событий между опросами.

Классическая гистограмма распределяет наблюдения по интервалам, buckets, с заданными верхними границами. Эти счетчики накопительные. Если запрос занял 0,2 секунды, он входит и в интервал до 0,5 секунды, и до 1 секунды, и в общий интервал +Inf. Также публикуются число наблюдений, count, и сумма значений, sum.

PROMQL

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(payment_processing_seconds_bucket[5m])
  )
)
```

Запрос вычисляет приближение p95 по всем выбранным экземплярам при одинаковых границах интервалов. Метрика учебная и должна реально экспортироваться. Для отдельных сервисов добавляют условия выбора нужных серий или группировку. Нельзя усреднять p95 каждого Pod и получать общий p95.

Summary с локальными процентилями обычно нельзя корректно агрегировать в общий процентиль нескольких экземпляров. Число наблюдений и сумма значений при этом агрегируются. Prometheus также поддерживает гистограммы нового формата, native histograms, с составными значениями и другой организацией интервалов. Их доступность зависит от всей цепочки инструментирования и хранения. Запрос выше относится именно к классической гистограмме.

P95 означает, что примерно 95% наблюдений укладывается в это значение. Точность гистограммы зависит от разрешения. Если интервалы слишком широкие около целевого времени ответа, оценка будет грубой. Среднее время не показывает редкие длинные запросы.

6.8 Для чего нужна графана, метрики в ней, алерты?

Grafana подключается к источникам данных и показывает результаты запросов. Для метрик это часто Prometheus, для логов и traces могут использоваться другие хранилища. Grafana сама по себе не является обязательным местом хранения всех метрик.

Хороший дашборд отвечает на ограниченный набор вопросов. Есть ли пользовательская проблема, какие операции страдают и на каком уровне искать причину? Сначала показывают успешность, задержку и поток операций, затем ресурсы и зависимости.

Алерт является регулярно проверяемым условием с политикой уведомления. Например, доля ошибок платежа удерживается выше допустимого уровня несколько минут. В уведомлении нужны сервис, масштаб проблемы, ссылка на наблюдения и инструкция первого действия.

Отсутствие данных нужно отличать от нуля ошибок. Если опрос приложения перестал проходить, график может стать пустым именно во время аварии. Правила должны учитывать отсутствие данных, ошибки опроса и ограничения источника.

Для снижения шума используют разумную длительность условия, группировку, подавление зависимых уведомлений при общей аварии и временное отключение уведомлений при известных работах. Можно применять правила Prometheus с Alertmanager или Grafana Alerting. Дублирование одинакового алерта в двух независимых системах обычно усложняет дежурство.

Метрики нужны для решений. Если алерт регулярно срабатывает и никто не должен ничего делать, его условие или назначение стоит пересмотреть.

7. Spring Cloud

7.1 Для чего Spring Cloud Config и как безопасно хранить секреты/ключи?

Spring Cloud Config отделяет настройки приложения от его сборки. Config Server является отдельным сервисом, который выдает эти настройки. Он может читать их, например, из Git или хранилища секретов Vault. Приложение запрашивает настройки по своему имени, профилю окружения и label. При хранении в Git label выбирает ветку, тег или конкретный коммит.

В Git удобно хранить несекретные параметры, историю изменений и проверку изменений коллегами. Один образ приложения можно использовать в разных окружениях, подставляя разные значения. Права на репозиторий и Config Server все равно должны соответствовать доступности конфигурации.

PROPERTIES

```
spring.application.name=payment-api
spring.config.import=configserver:https://config.internal
spring.cloud.config.label=release-2026-11
```

Фрагмент application.properties относится к Spring Boot 4.x и Spring Cloud Config 5.x с Config Client в зависимостях. Указанная ветка или метка должна существовать в хранилище настроек. Адрес учебный. Данные доступа и доверенные сертификаты для TLS передаются безопасным способом отдельно.

Начиная со Spring Boot 2.4 импорт настроек через Config Data стал основным способом подключения. Bootstrap.yml не является обязательным для этого пути. Префикс `optional:` разрешает продолжить запуск при недоступности источника, если остальной конфигурации достаточно. Без него ошибка загрузки останавливает запуск. Для критичных параметров решение о таком поведении принимают явно.

Обычный Git не является хранилищем открытых секретов. Для них используют Vault или другое хранилище секретов, минимальные права и ротацию. Шифрование значений Config Server помогает защитить представление в репозитории, но приложение все равно получает расшифрованный секрет, а серверу нужен доступ к ключу.

Начальные данные доступа к самому Config Server тоже нужно откуда-то получить. Они не могут находиться только внутри конфигурации, которую без них невозможно загрузить. Не допускают утечки через URL, логи, Actuator и историю Git.

7.2 Для чего нам вообще Config как отдельный сервис?

Когда приложений мало, конфигурации в манифестах развертывания и переменных окружения может быть достаточно. Отдельный Config Server нужен, когда централизованное управление дает понятный выигрыш.

Он предоставляет единый механизм выбора параметров по приложению, окружению и версии. Команда может проверить изменение, выбрать прежнюю версию настроек и видеть, какая конфигурация должна использоваться. При этом слишком общий файл со всеми параметрами всех сервисов способен стать новым источником связанности.

Config Server обычно нужен при запуске, а не при каждом бизнес-запросе. Загруженные настройки попадают в Spring Environment, объект с набором доступных приложению значений. Затем Spring создает и настраивает бины, то есть управляемые им объекты. Изменение файла в Git не означает, что уже созданные объекты сразу получат новые значения.

Для обновления настроек без перезапуска нужны поддерживающие это механизмы. Такое обновление называют refresh. Не каждый бин можно безопасно пересоздать на ходу. Изменить простой флаг и заменить работающий пул соединений с базой являются разными задачами. Иногда надежнее постепенно перезапустить экземпляры приложения.

Централизация добавляет зависимость при старте и массовых перезапусках. Нужны доступность сервера, резервирование хранилища настроек и проверенный сценарий недоступности. Для воспроизводимого релиза полезно связывать версию приложения с проверенной версией конфигурации.

Если Kubernetes ConfigMap, Secret и выбранный сервис управления секретами уже закрывают задачу, Config Server не обязателен только потому, что проект использует Spring Cloud.

7.3 Как настроить Spring Cloud Gateway: маршрутизация, фильтры, троттлинг?

Spring Cloud Gateway принимает запрос клиента, выбирает сервис и передает запрос ему. Правило передачи называется маршрутом. У маршрута есть имя, адрес сервиса, условия выбора и фильтры. Условия, predicates, проверяют, например, домен и путь запроса. Фильтры выполняют дополнительные действия, например проверяют доступ или меняют путь перед отправкой.

Ниже пример для **Gateway Server WebFlux 5.0.x**, совместимого с линейкой Spring Cloud 2025.1 и соответствующим Spring Boot 4.x. Нужен стартер, зависимость с готовой интеграцией Spring, `spring-cloud-starter-gateway-server-webflux`. Поддержка Boot 4.1 начинается с Cloud 2025.1.2. Для ограничителя частоты запросов через Redis требуется `spring-boot-starter-data-redis-reactive` и доступный Redis.

YAML

```yaml
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: orders
              uri: http://orders:8080
              predicates:
                - Path=/api/orders/**
              filters:
                - StripPrefix=1
                - name: RequestRateLimiter
                  args:
                    key-resolver: "#{@userKeyResolver}"
                    redis-rate-limiter.replenishRate: 10
                    redis-rate-limiter.burstCapacity: 20
```

Запрос `/api/orders/42` подходит под условие Path и попадает в маршрут orders. Фильтр StripPrefix удаляет первый сегмент пути, api. Поэтому сервис заказов получит `/orders/42`. Имя orders должно превращаться в доступный сетевой адрес, например через Service в Kubernetes. Запись `lb://orders` означает выбор экземпляра через Spring Cloud LoadBalancer. Для нее настраивают обнаружение адресов сервисов.

KeyResolver определяет, к кому относится ограничение частоты. Например, он берет имя уже проверенного пользователя, чтобы у каждого была своя квота. Это не отдельная проверка пароля. Код ниже ожидает, что аутентификация уже выполнена.

```java
@Bean
KeyResolver userKeyResolver() {
    return exchange -> exchange.getPrincipal()
        .map(Principal::getName);
}
```

Нужны импорты `Bean`, `java.security.Principal` и `org.springframework.cloud.gateway.filter.ratelimit.KeyResolver`, а также настроенная аутентификация. Взять userId из произвольного параметра URL небезопасно, клиент сможет обходить квоту.

Ограничитель через Redis использует алгоритм корзины токенов, token bucket. В корзину регулярно добавляются разрешения, а каждый запрос забирает одно. Здесь токены добавляются со скоростью 10 в секунду, емкость равна 20, стоимость запроса по умолчанию 1. Короткий всплеск допустим, устойчивое превышение ведет к 429. Поведение при недоступности Redis проверяют отдельно.

В старых примерах встречается другой префикс свойств и прежний стартер. Копировать настройки между основными версиями без проверки нельзя. Gateway Server Web MVC является отдельным вариантом с другой моделью выполнения.

7.4 Как использовать OpenFeign: таймауты, ретраи, декодеры ошибок, fallback?

OpenFeign создает HTTP-клиент по Java-интерфейсу. Приложение вызывает его метод, а библиотека формирует запрос и читает ответ. Encoder превращает аргументы в отправляемое тело. Decoder разбирает успешный ответ. ErrorDecoder обрабатывает ошибочный HTTP-ответ и превращает его в исключение, с которым работает Java-код.

Для Spring Cloud OpenFeign 5.x нужны стартер OpenFeign и включение поиска клиентских интерфейсов, обычно через `@EnableFeignClients`. Пример интерфейса можно ограничить одним методом.

```java
@FeignClient(name = "payments", url = "${payments.url}")
interface PaymentsClient {
    @GetMapping("/payments/{id}/status")
    String getStatus(@PathVariable("id") String id);
}
```

Здесь используются Spring Cloud `FeignClient` и Spring Web `GetMapping`, `PathVariable`. Пример предполагает, что эндпоинт возвращает обычную строку. Для JSON-модели выбирают соответствующий тип ответа. URL задается в конфигурации.

YAML

```yaml
spring:
  cloud:
    openfeign:
      client:
        config:
          payments:
            connectTimeout: 500
            readTimeout: 1500
            loggerLevel: basic
```

Значения таймаутов в этой конфигурации задаются в миллисекундах. Это не общий предел времени всей бизнес-операции. Поиск адреса по имени через DNS, получение соединения из пула, TLS и дополнительные попытки зависят от используемого HTTP-клиента и его настроек.

Spring Cloud OpenFeign по умолчанию создает Retryer.NEVER_RETRY. Это отличается от поведения базового Feign. Ретраи включают осознанно, согласуют с внешней защитой и разрешают только при безопасной семантике операции.

ErrorDecoder должен отличать бизнес-отказ от временного сбоя. Например, отсутствие денег и недоступность платежного сервиса требуют разной реакции. Fallback задает запасное поведение при отказе и работает при соответствующей интеграции с Circuit Breaker. FallbackFactory позволяет учитывать причину ошибки. Запасной ответ может вернуть допустимые данные из кэша, но не должен выдавать неизвестный результат платежа за успех.

Проект Spring Cloud OpenFeign считается завершенным по набору основных возможностей, feature-complete. Он продолжает сопровождаться, но для новых разработок команда Spring предлагает рассматривать Spring HTTP Service Clients. Это не означает, что существующие Feign-клиенты необходимо срочно переписать без причины.

7.5 Как включать CircuitBreaker (Resilience4j) и какие параметры критичны?

Resilience4j реализует Circuit Breaker. Как он прекращает вызовы неисправной зависимости, разобрано в разделе паттернов. Spring Cloud CircuitBreaker предоставляет общий способ подключения этого механизма к Spring-приложению. Для обычных синхронных вызовов используют `spring-cloud-starter-circuitbreaker-resilience4j`. Совместимые версии зависимостей задает Spring Cloud BOM, общий список согласованных версий библиотек. Для реактивного приложения существует отдельный стартер.

Resilience4JCircuitBreakerFactory создает настроенный ограничитель для конкретной зависимости. Например, имя payments связывает его с настройками оплаты. Сам вызов нужно передать этому ограничителю, иначе он о нем не узнает. В коде ниже `factory` является такой фабрикой, а `client` клиентом PaymentsClient из предыдущего вопроса.

```java
return factory.create("payments")
    .run(() -> client.getStatus(paymentId));
```

Здесь paymentId является аргументом метода, а ошибка без предусмотренного запасного поведения продолжает быть ошибкой. Фабрика сама по себе не превращает любой вызов в приложении в защищенный.

```yaml
resilience4j:
  circuitbreaker:
    instances:
      payments:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 50
        minimumNumberOfCalls: 20
        failureRateThreshold: 50
        slowCallDurationThreshold: 800ms
        slowCallRateThreshold: 70
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 3
```

Это учебные параметры для разбора поведения. При двадцати наблюдениях порог ошибок начинает иметь смысл, окно ограничено последними пятьюдесятью вызовами. Отдельно настраивают исключения, считающиеся отказом, и исключаемые бизнес-ситуации.

Порог длительности медленного вызова отмечает длительность для статистики, а не прерывает вызов на 800 мс. Сетевые таймауты и TimeLimiter, механизм ограничения времени ожидания результата, настраиваются отдельно. Для блокирующего кода попытка отмены Future тоже не гарантирует остановку внешней операции.

Есть и подключение через аннотации. Spring создает вокруг объекта прокси, промежуточную оболочку, которая перехватывает вызов метода и применяет Circuit Breaker. Это использование AOP, механизма добавления общего поведения вокруг методов. Если объект вызывает свой метод напрямую через this, оболочка может не участвовать. Поэтому аннотация при таком внутреннем вызове может не сработать. Несколько механизмов повторов и ограничений времени нужно согласовывать между собой.

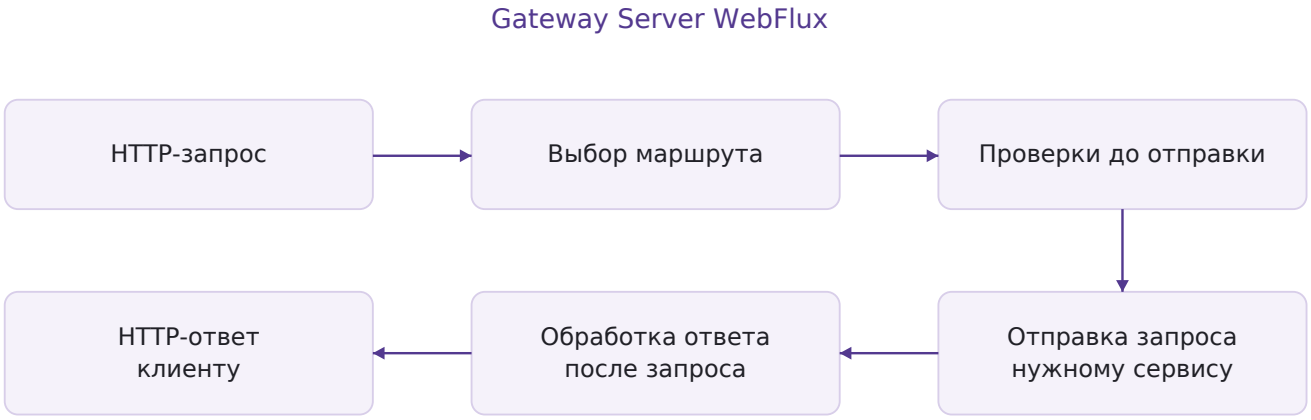
7.6 Как под капотом работает Spring Cloud Api Gateway? Что принимает, что возвращает

Клиент отправляет в Spring Cloud Gateway HTTP-запрос. В нем есть метод, адрес, заголовки и при необходимости тело. Gateway выбирает нужный сервис и передает запрос ему. Полученный ответ со статусом, заголовками и телом он возвращает клиенту. Поэтому со стороны клиента Gateway является сервером, а со стороны внутреннего сервиса HTTP-клиентом.

В варианте на WebFlux сведения о запросе и будущем ответе находятся в объекте ServerWebExchange. Компонент Handler Mapping определяет, кто должен обработать запрос. В Gateway его задача состоит в выборе маршрута. Он проверяет настроенные условия, например «путь начинается с /api/orders», и находит подходящее правило передачи запроса.

После выбора маршрута компонент Gateway Web Handler собирает цепочку фильтров. Фильтр является небольшим обработчиком до или после обращения к сервису. GlobalFilter применяется ко всем маршрутам, а GatewayFilter к конкретному маршруту. Их порядок определяет, какая проверка или изменение выполнится раньше.

До отправки фильтры могут проверить права пользователя, ограничить частоту запросов, изменить путь и заголовки. Затем фильтр отправки обращается к внутреннему сервису через HTTP-клиент, обычно Reactor Netty HttpClient. Когда ответ получен, выполняется обратная часть цепочки фильтров. Она может изменить ответ или записать длительность запроса.



Фильтр может завершить запрос до обращения к сервису

Тело можно передать частями, не собирая целиком в объект

Фильтр может сам закончить обработку. Например, вернуть 401 без входа пользователя или 429 при превышении квоты. Тогда запрос до сервиса вообще не дойдет. Действия после ответа обычно выполняются в обратном порядке относительно действий до запроса. Если один фильтр начал измерять время, а другой отправил запрос, первый закончит измерение уже после возвращения ответа.

Gateway не обязан превращать каждое тело запроса в Java-объект. Проксирование может работать с потоком буферов. Чтение и изменение тела требует отдельной обработки, ограничений размера и корректного управления памятью. Бесконтрольное накопление крупных тел запросов способно перегрузить gateway.

WebFlux может обслуживать много соединений небольшим числом потоков. Поток не должен просто ждать завершения сетевого обращения, он переключается на доступную работу. Такие рабочие потоки называют event loop. Если выполнить в них блокирующий запрос к базе через JDBC или долгий Feign-вызов, поток будет занят

ожиданием и задержит другие соединения. Такую работу выполняют с подходящим клиентом или в отдельно выделенных потоках.

Gateway Server Web MVC построен на другом механизме обработки запросов. Поэтому описание потоков WebFlux нельзя переносить на него буквально. В обоих вариантах внутренние сервисы проверяют свои бизнес-правила и права доступа. Если Gateway передает имя пользователя в заголовке, сервис должен доверять этому заголовку только от защищенной входной точки, а не от произвольного клиента.